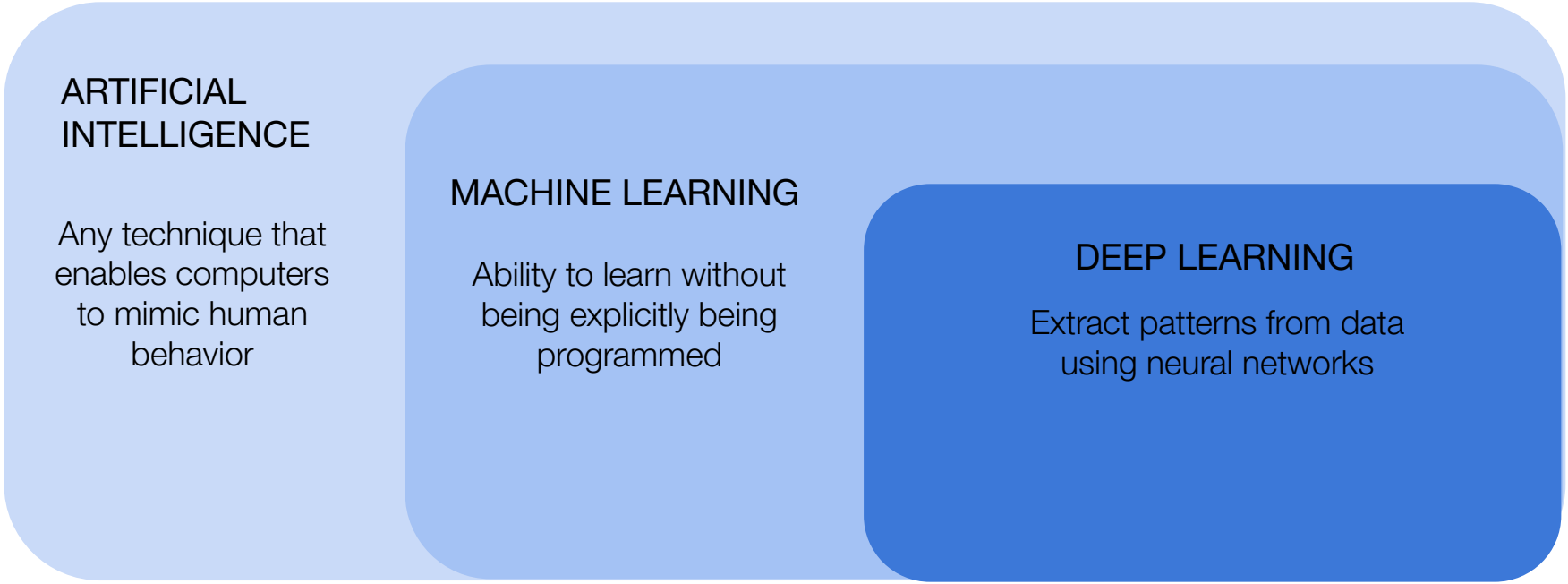
Introduction to Deep Learning

JENNIFER JOYCE M. MONTEMAYOR - MAULANA

Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology

What is Deep Learning?



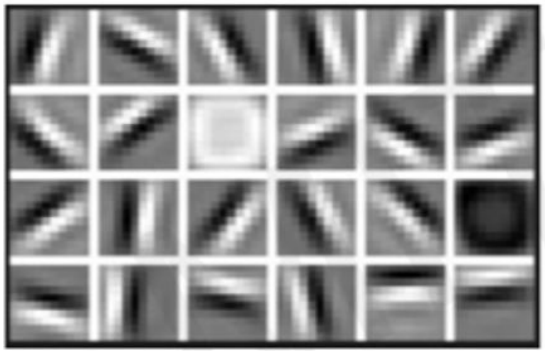
Teach computers how to **learn a task** directly from **raw data**

Why Deep Learning?

Manually engineered features are time consuming and not scalable in practice.

Can we learn the **underlying features** directly from data?

Low level features



Lines and edges

Mid level features



Eyes, nose, ears

High level features

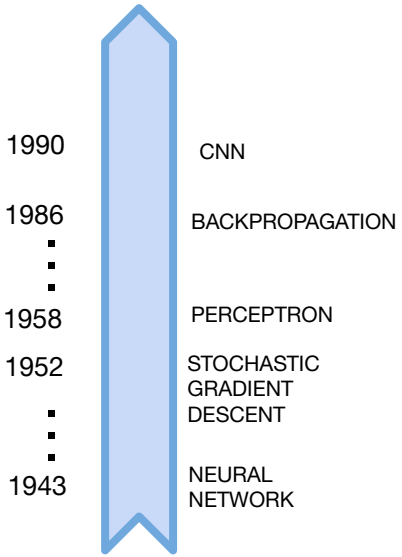


Facial structure

Why now?

Neural Networks are decades old...

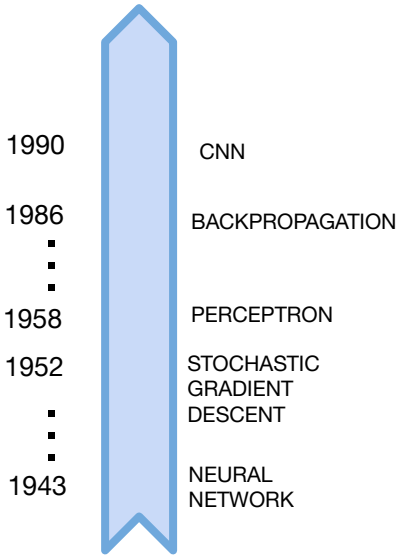
- In 1949, neurophysiologist Warren McCulloch and mathematician Walter Pitts wrote "*A Logical Calculus of the Ideas Immanent in Nervous Activity*" on how neurons might work
- In 1990, computer scientist Yann André LeCun implemented Convolutional Neural Networks (CNN) in their paper "*Gradient-Based Learning Applied to Document Recognition*"
- In 1997, Long Short-Term Memory (LSTM) was developed by Sepp Hochreiter



Why the resurgence?

Why now?

"If Deep Learning is the steam engine of the AI revolution, then data is its coal: the raw material that powers our intelligent machines, without which nothing would be possible"



Big Data

- Larger datasets
- Easier collection and storage
  - The rise of the internet make it feasible to collect and distribute very large datasets

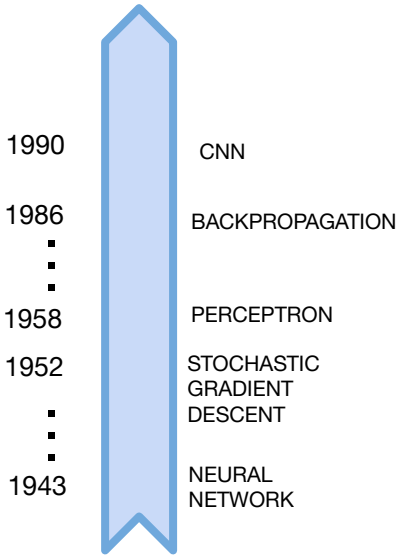
IMAGENET

14,197,122 images



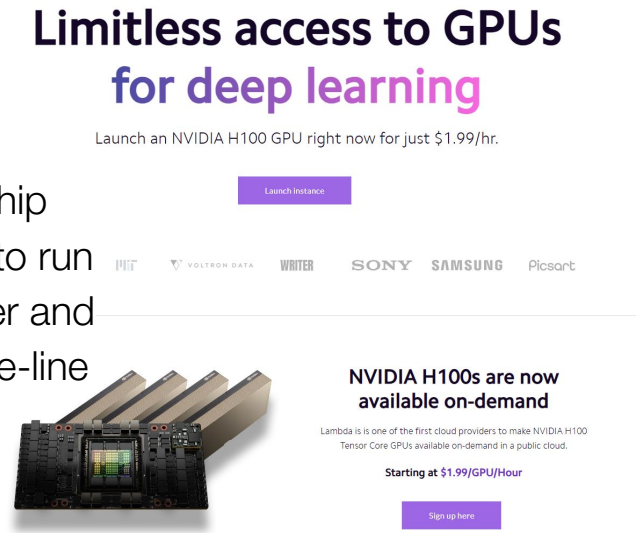


Why now?

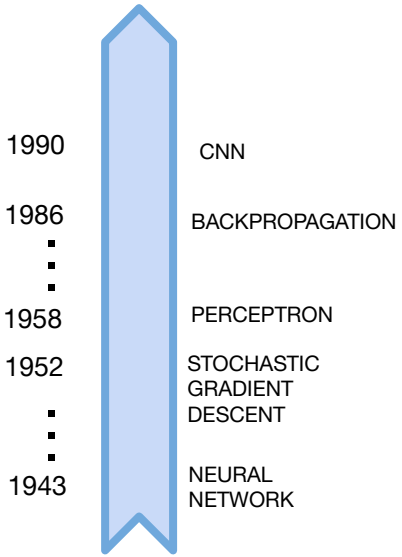


Hardware

- gaming market subsidized supercomputing for the next generation of artificial intelligence applications
- Massively parallel chips
- Graphics Processing Units (GPUs)
- In 2016, Google revealed its Tensor Processing Unit (TPU) project: a new chip design developed from the ground up to run deep neural networks significantly faster and far more energy efficient than top-of-the-line GPUs



Why now?



Software

- Algorithms
  - advent of several simple but important algorithmic improvements that allowed for better gradient propagation:
    - activation functions for neural layers
    - weight initialization schemes
    - optimization schemes
- Toolboxes

Keras

TensorFlow

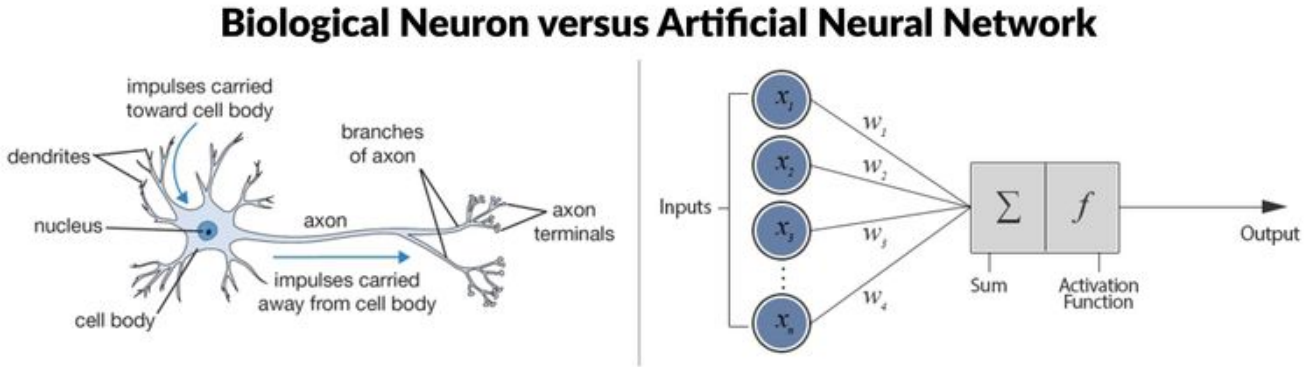
PyTorch

The Perceptron

The structural building block of deep learning

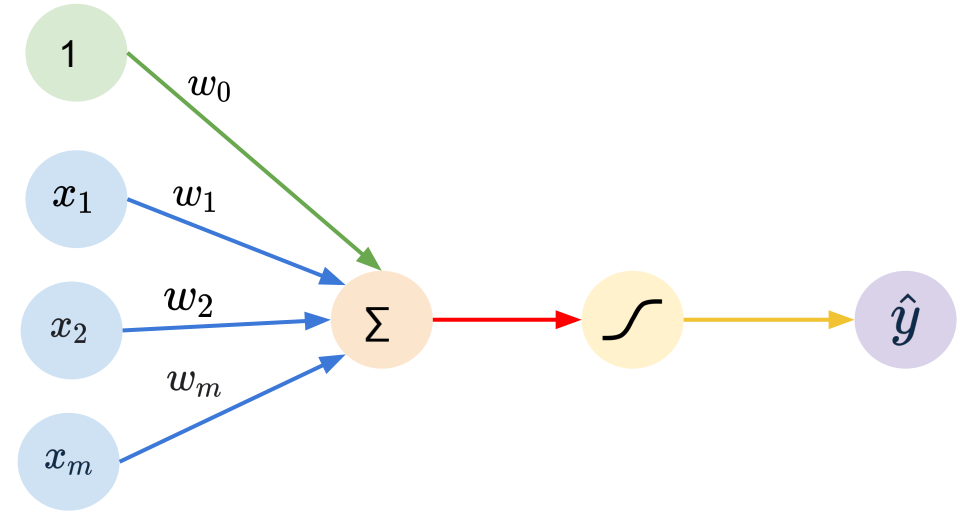
Artificial Neural Network (ANN)

- computational model inspired by the way biological neural networks in the human brain process information
- ability to learn the mapping function from observed / training data
- predictive capability comes from the hierarchical or multi-layered structure of the network



Perceptron: Forward Propagation

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



LINEAR COMBINATION OF INPUTS

NON-LINEAR ACTIVATION FUNCTION

$$\hat{y} = g(w_0 + \sum_{i=1}^{m} x_i \ w_i)$$

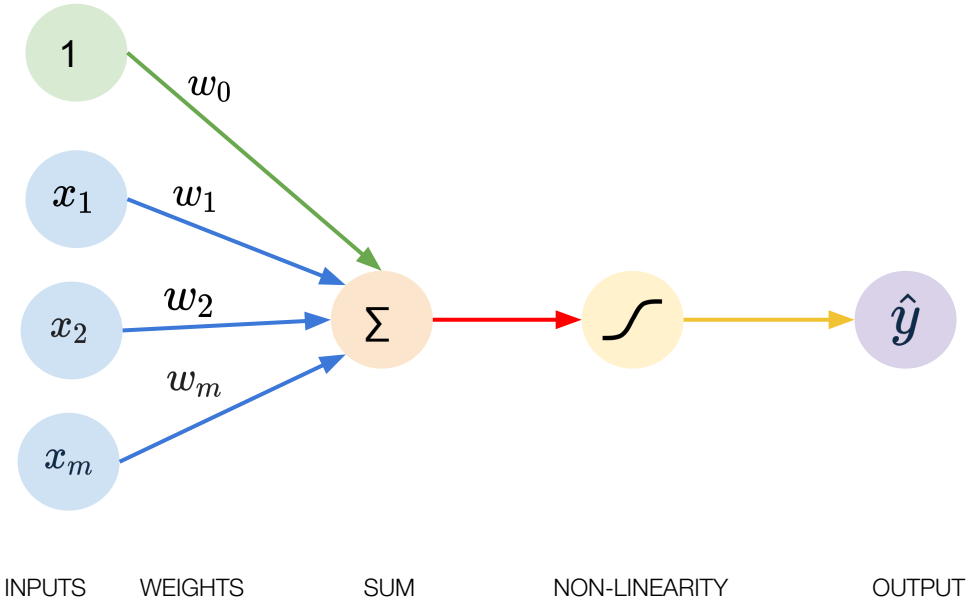
BIAS

INPUTS

WEIGHTS

Perceptron: Forward Propagation

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”

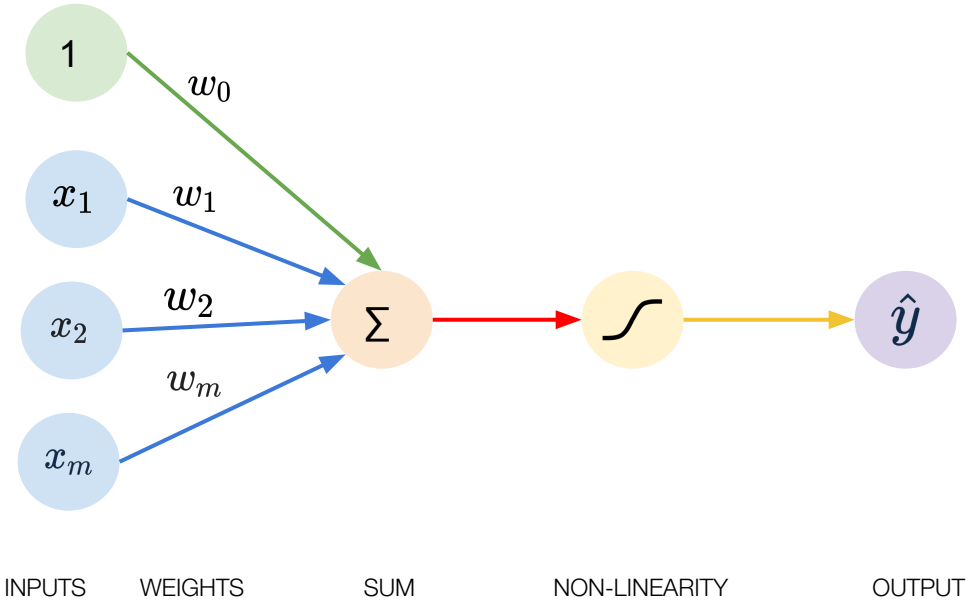


$$\hat{y} = g(\, w_0 + X^T \, W)$$

where $X = \begin{bmatrix} x_1 \\ \cdot \\ \cdot \\ x_m \end{bmatrix}$ and $W = \begin{bmatrix} w_1 \\ \cdot \\ \cdot \\ w_m \end{bmatrix}$

Perceptron: Forward Propagation

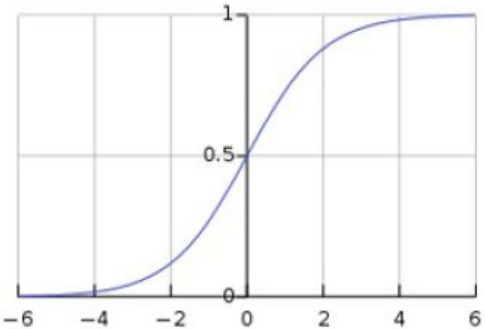
- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



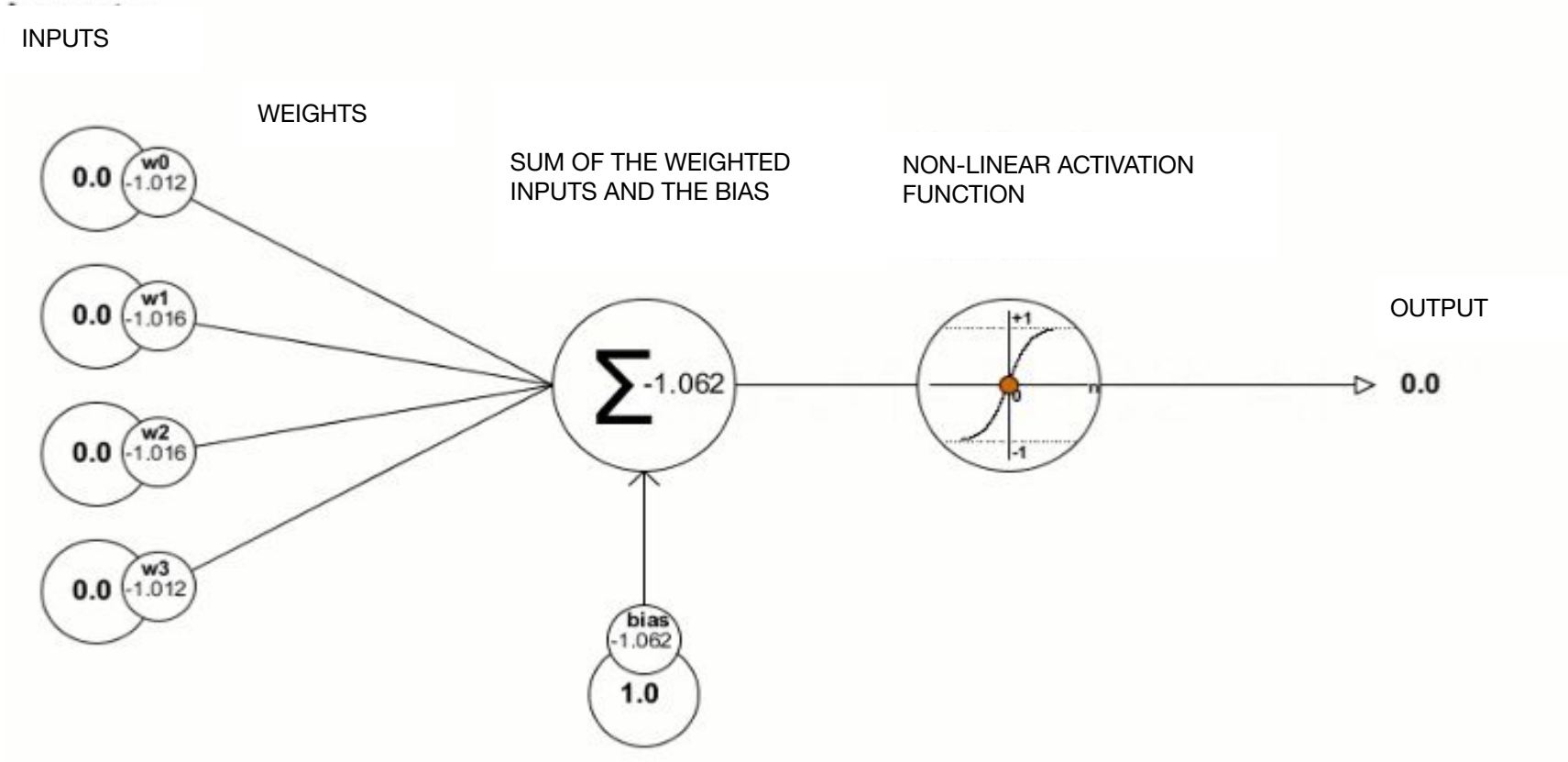
$$\hat{y} = g(w_0 + X^T W)$$

Example: sigmoid function

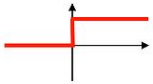
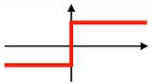
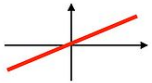
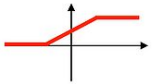
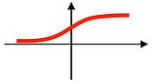
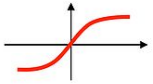
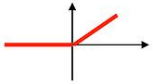
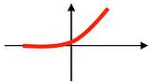
$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Perceptron: Forward Propagation



Common Activation Functions

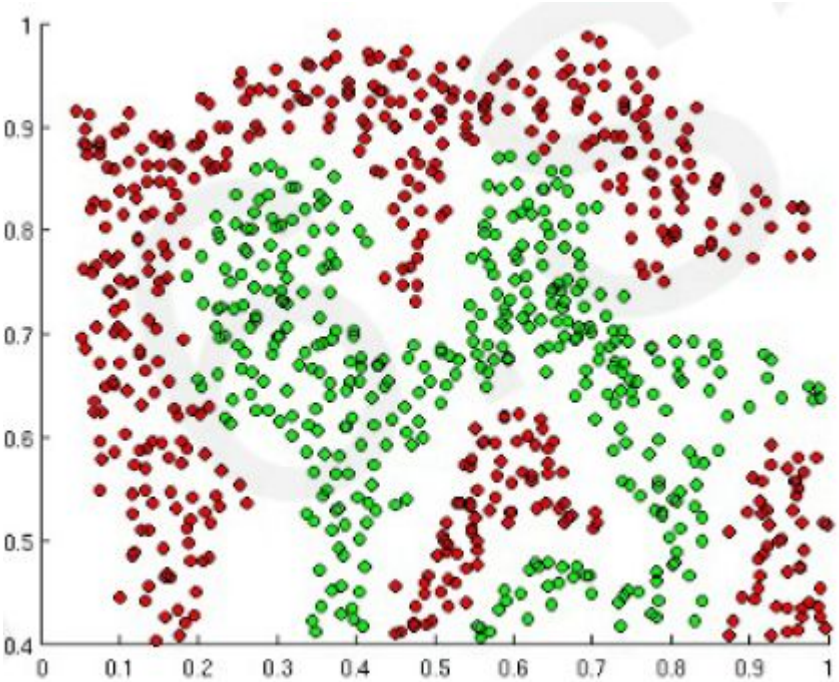
Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^{z})$	Multi-layer Neural Networks	

Copyright © Sebastian Raschka 2016 (http://sebastianraschka.com)

Activation Function

- Non-linear
  - model patterns in dataset are non-linear
- Continuously differentiable
  - to improve model using gradient descent, output must have a nice slope so error derivatives with respect to weights can be computed
- Fixed Range
  - squash the input data into a narrow range that

Importance of Activation Functions

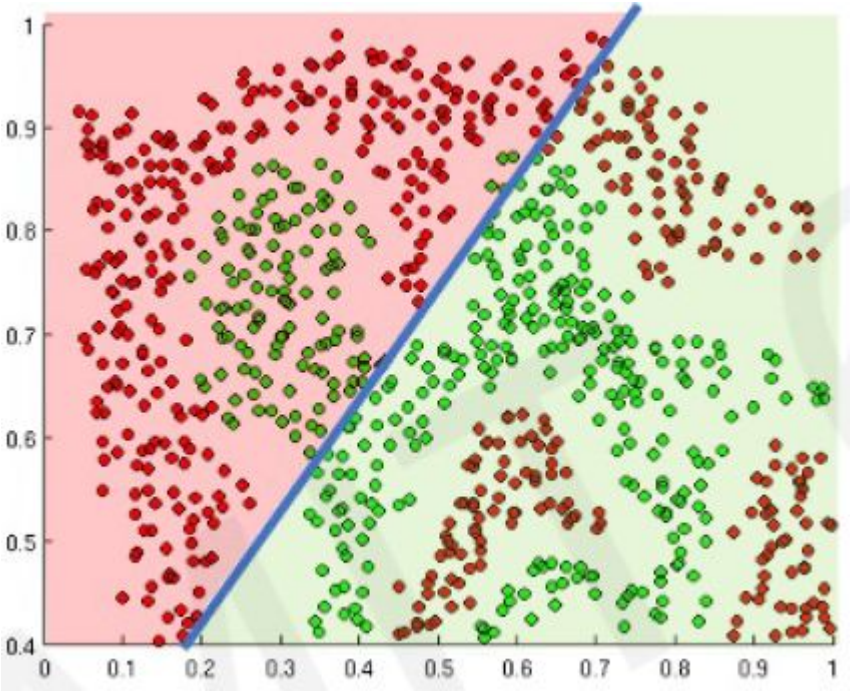


The purpose of activation functions is to introduce non-linearities into the network.

Without an activation function, the output will always be linear.

Non-linearities allows us to approximate arbitrarily complex functions.

Importance of Activation Functions

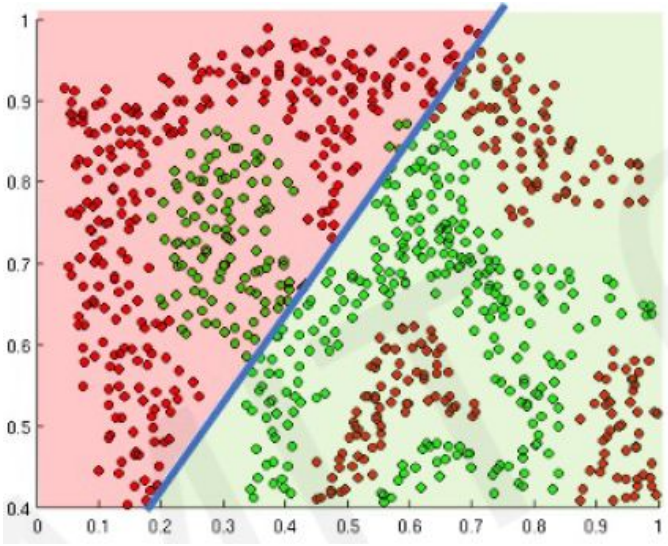


The purpose of activation functions is to introduce non-linearities into the network.

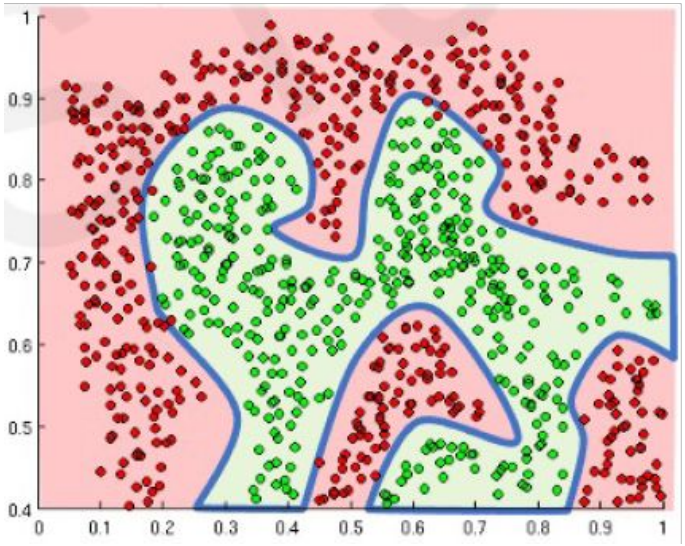
Without an activation function, the output will always be linear.

Non-linearities allows us to approximate arbitrarily complex functions.

Importance of Activation Functions



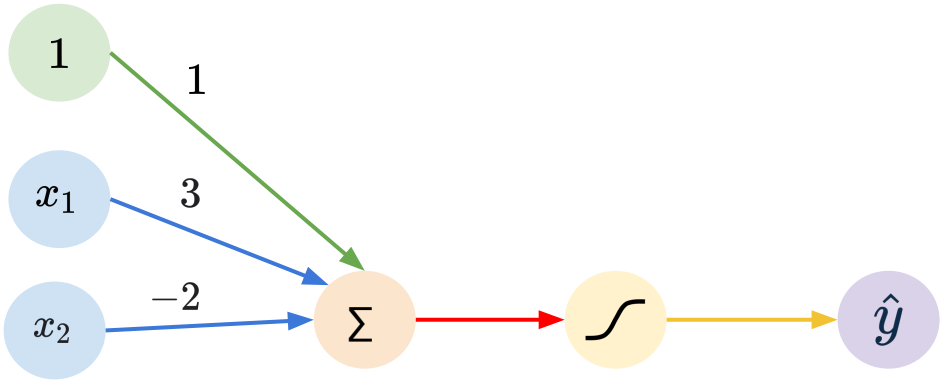
Linear activations produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Perceptron: An example

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



INPUTS WEIGHTS SUM NON-LINEARITY OUTPUT

We have: $w_0 = 1$ and $W = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

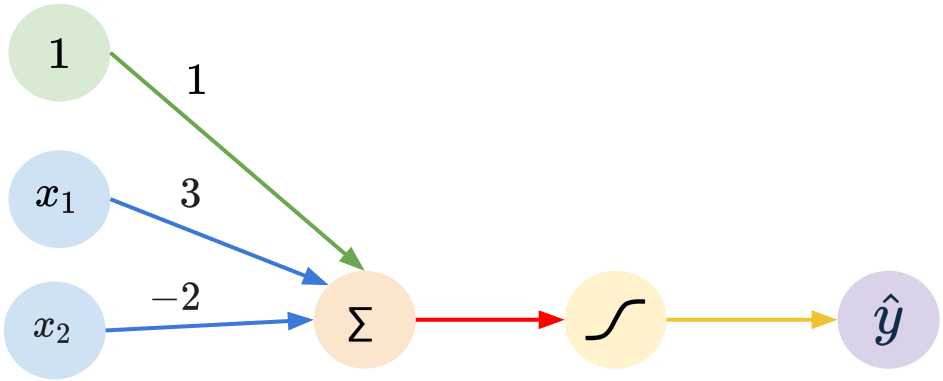
$$\hat{y} = g(\, w_0 + X^T \, W)$$

$$\hat{y} = g(\, 1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix})$$

$$\hat{y} = g(\, 1 + 3x_1 - 2x_2)$$

Perceptron: An example

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



INPUTS WEIGHTS SUM NON-LINEARITY OUTPUT

We have: $w_0 = 1$ and $W = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$

$$\hat{y} = g(\, w_0 + X^T \, W)$$

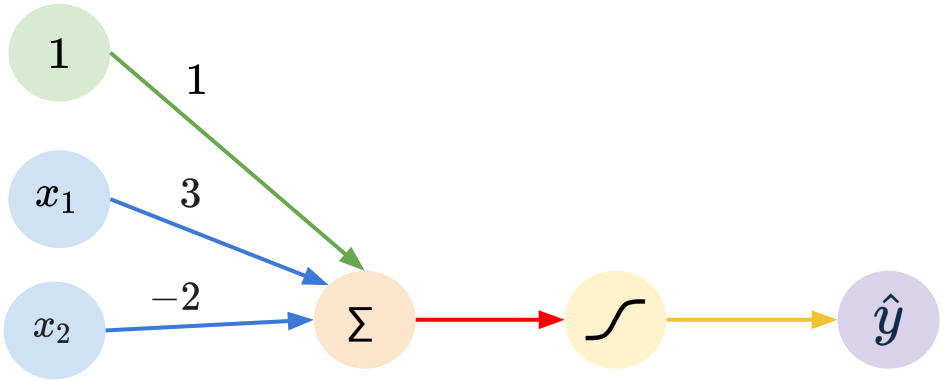
$$\hat{y} = g(\, 1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix})$$

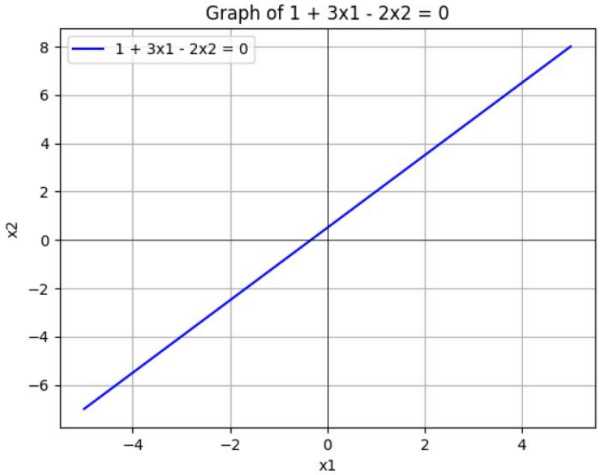
$$\hat{y} = g(\, 1 + 3x_1 - 2x_2)$$

This is a 2d line.

Perceptron: An example

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”

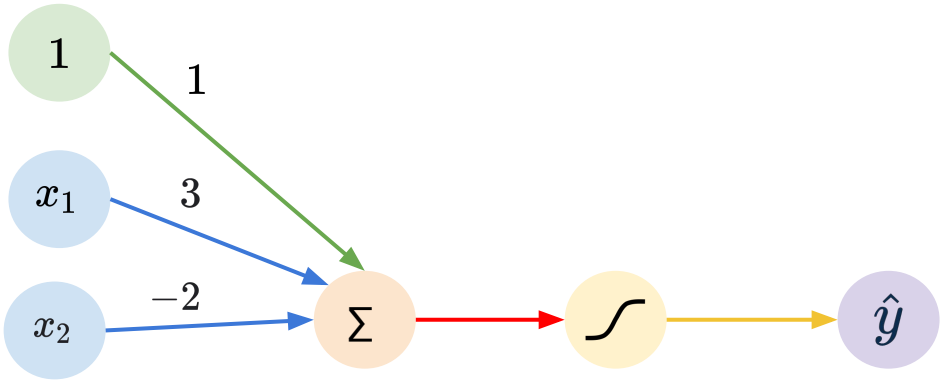




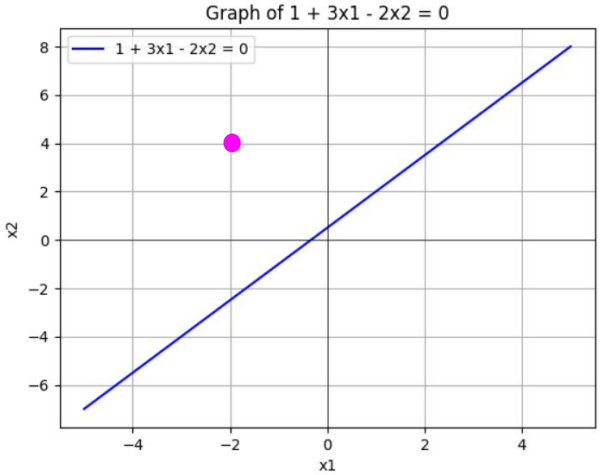
INPUTS WEIGHTS SUM NON-LINEARITY OUTPUT

Perceptron: An example

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



INPUTS WEIGHTS SUM NON-LINEARITY OUTPUT

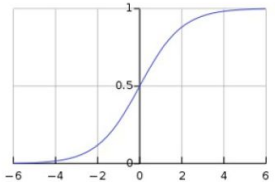


Assume we have input $X = \begin{bmatrix} -2 \\ 4 \end{bmatrix}$

$$\hat{y} = g(\ 1 + (3 * -2) - (2 * 4))$$

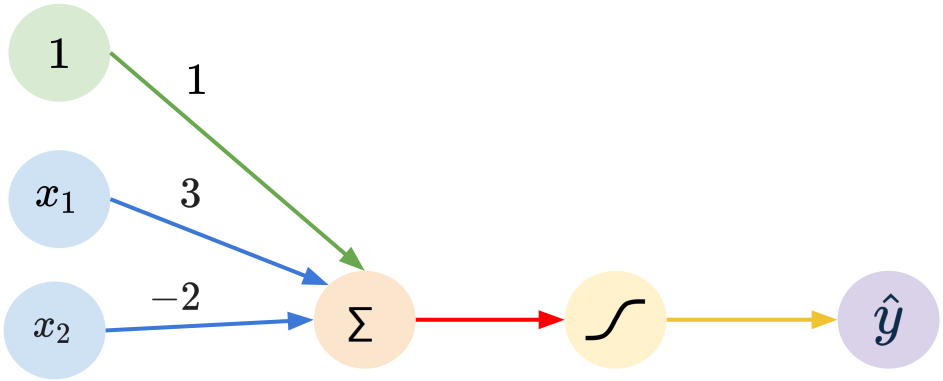
$$\hat{y} = g(-13)$$

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

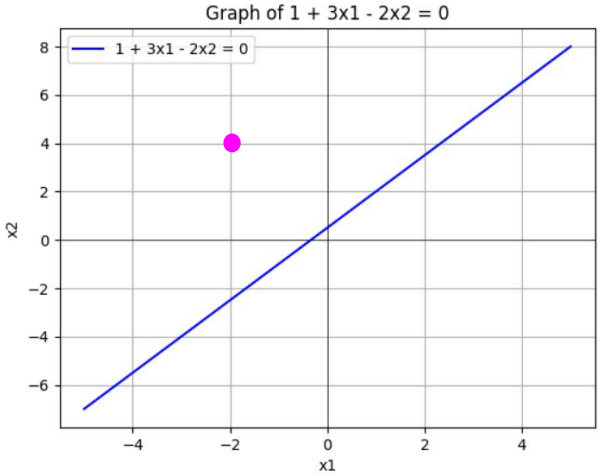


Perceptron: An example

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



INPUTS WEIGHTS SUM NON-LINEARITY OUTPUT



Assume we have input $X = \begin{bmatrix} -2 \\ 4 \end{bmatrix}$

$$\hat{y} = g(\ 1 + (3 * -2) - (2 * 4))$$

$$\hat{y} = g(-13)$$

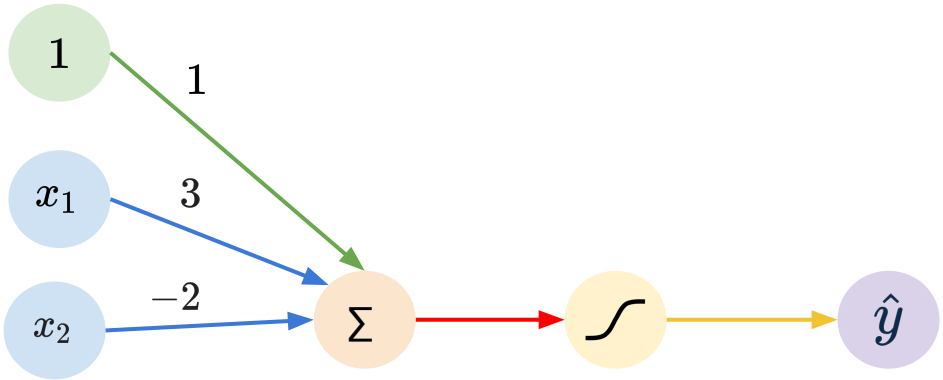
$$\hat{y} \approx 0.000002260$$

```
1  import numpy as np
2  def sigmoid(x):
3      return 1 / (1 + np.exp(-x))
4  sigmoid(-13)
```

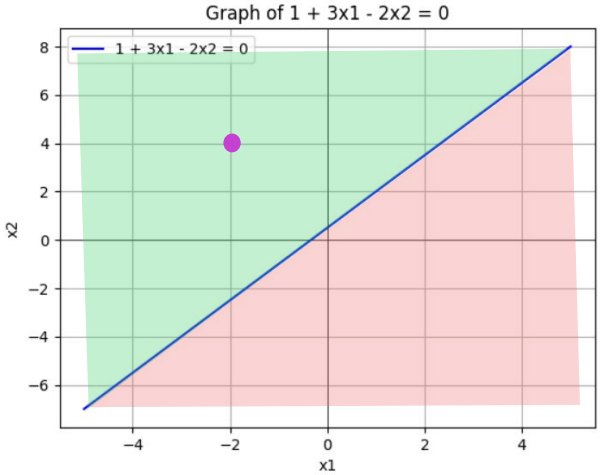
```
2.2603242979035746e-06
```

Perceptron: An example

- basic unit of computation in a neural network
- also called an “artificial neuron”, “node”, or “unit”



INPUTS WEIGHTS SUM NON-LINEARITY OUTPUT



Assume we have input $X = \begin{bmatrix} -2 \\ 4 \end{bmatrix}$

$$\hat{y} = g(\ 1 + (3 * -2) - (2 * 4))$$

$$\hat{y} = g(-13)$$

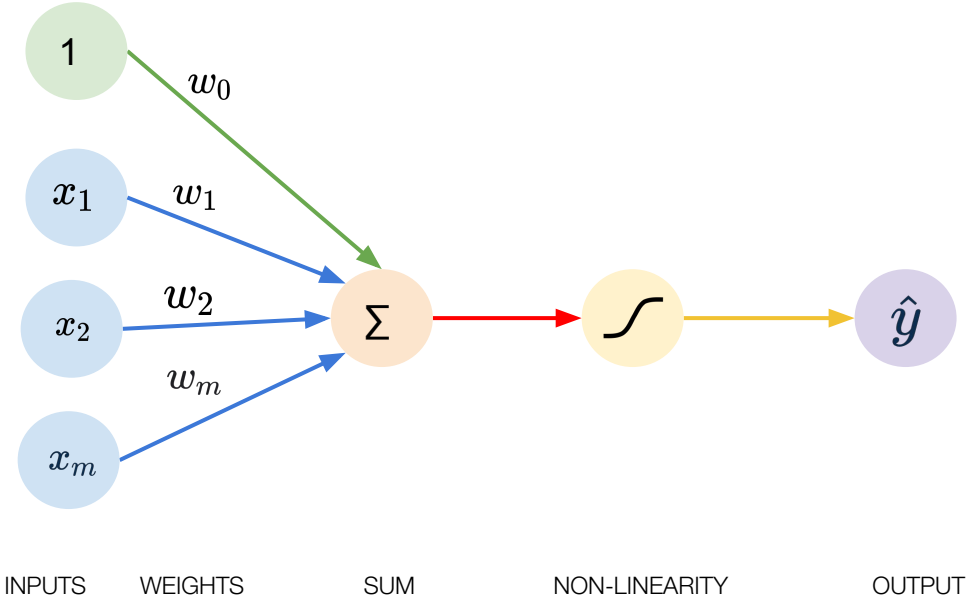
$$\hat{y} \approx 0.000002260$$

```
1  import numpy as np
2  def sigmoid(x):
3      return 1 / (1 + np.exp(-x))
4  sigmoid(-13)
```

```
2.2603242979035746e-06
```

Building Neural Networks with Perceptrons

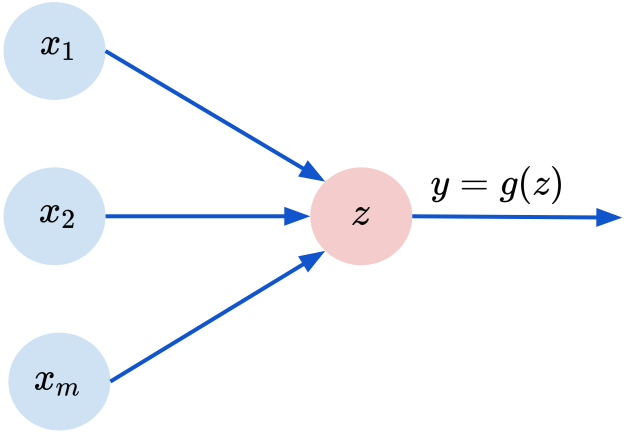
Perceptron: Simplified



$$\hat{y} = g(\ w_0 + X^T\ W)$$

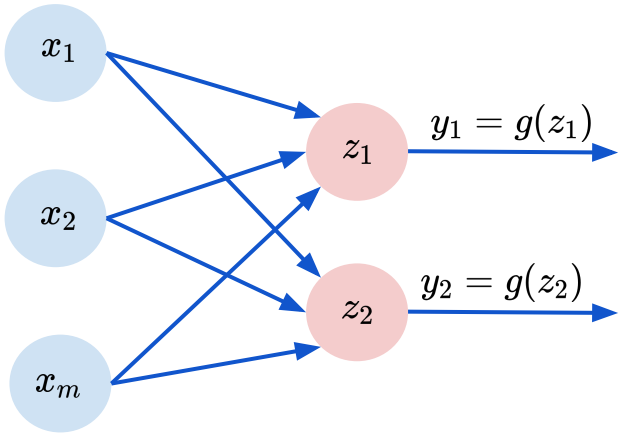
where $X = \begin{bmatrix} x_1 \\ \cdot \\ \cdot \\ x_m \end{bmatrix}$ and $W = \begin{bmatrix} w_1 \\ \cdot \\ \cdot \\ w_m \end{bmatrix}$

Perceptron: Simplified



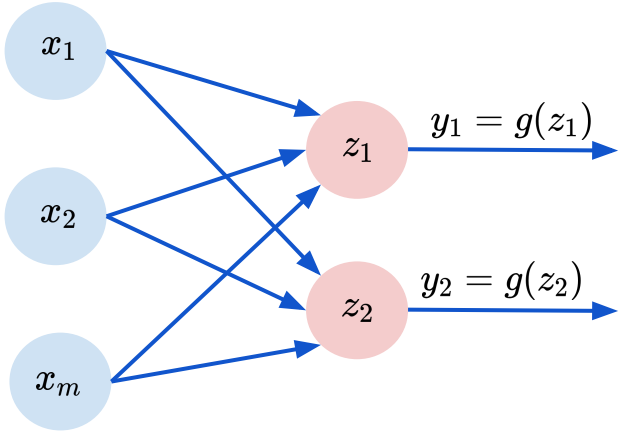
$$z = w_0 + \sum_{j=1}^{m} x_j \;\; w_j$$

Multi Output Perceptron



$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \;\; w_{j,i}$$

Multi Output Perceptron

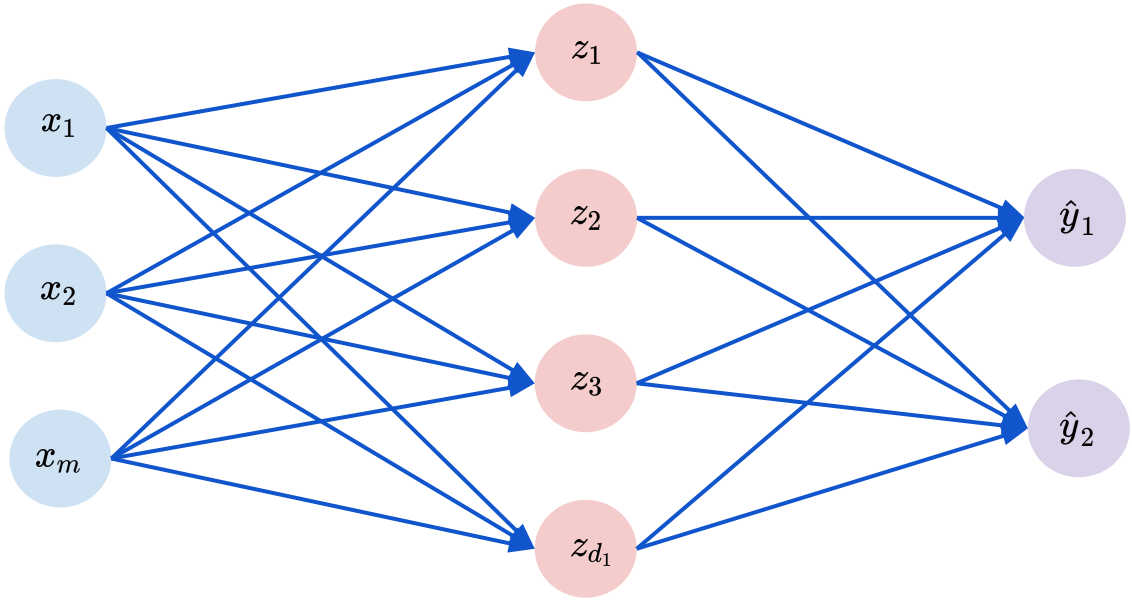


$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \;\; w_{j,i}$$

```
import tensorflow as tf

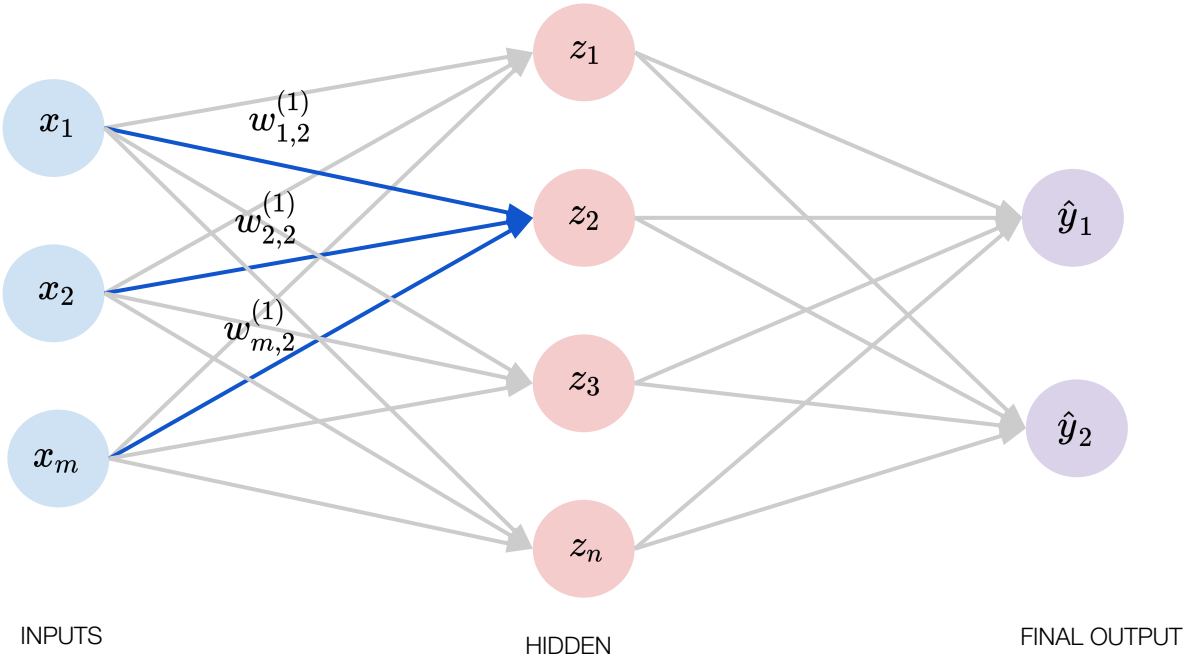
layer = tf.keras.layers.Dense(units = 2)
```

Single Layer Neural Network



$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \;\; w_{j,i} \qquad \hat{y}_i = g(w_{0,i} + \sum_{j=1}^{d_1} g(z_j) \;\; w_{j,i})$$

Single Layer Neural Network



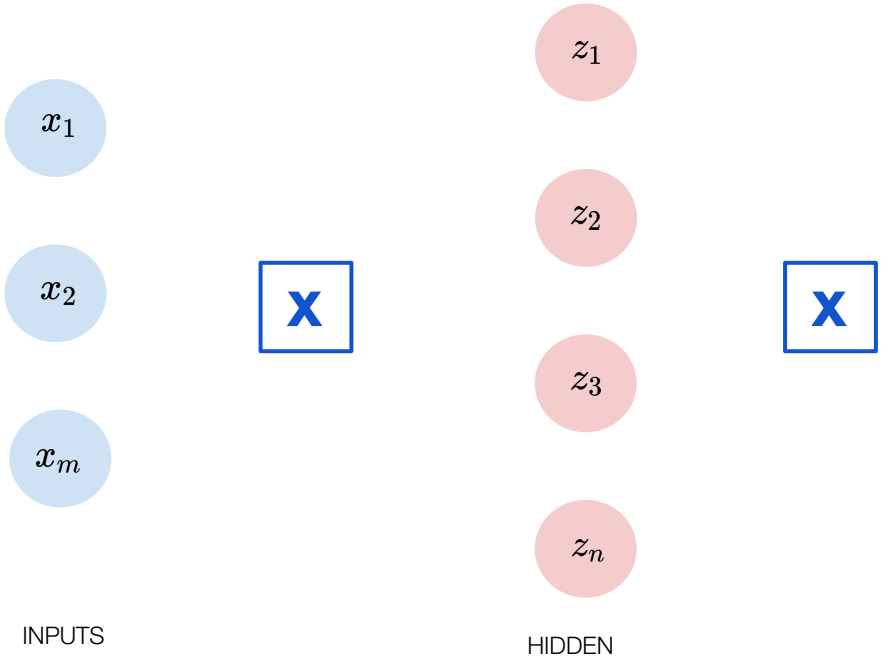
$$z_2 = w_{0,2}^{(1)} + \sum_{j=1}^{m} x_j w_{j,2}^{(1)}$$

$$z_2 = w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)}$$

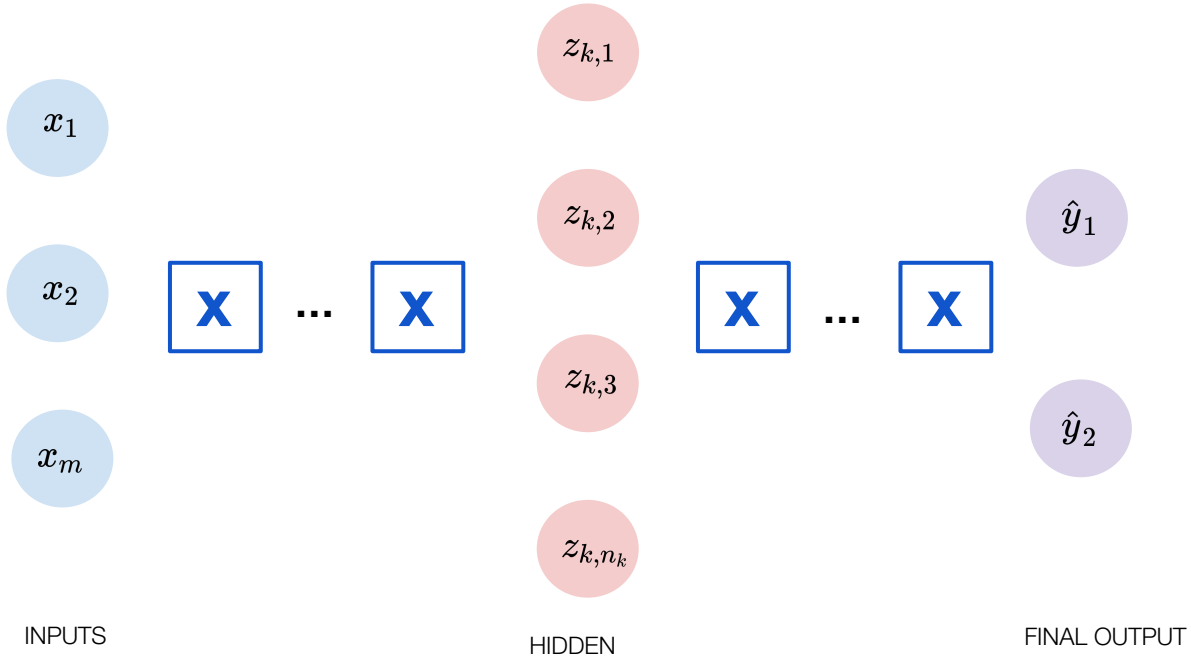
Multi Output Perceptron

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(n),
    tf.keras.layers.Dense(2)
])
```



Multi Output Perceptron



$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{(k-1)}, j) w_{j,i}^{(k)}$$

Applying Neural Networks

Example Problem

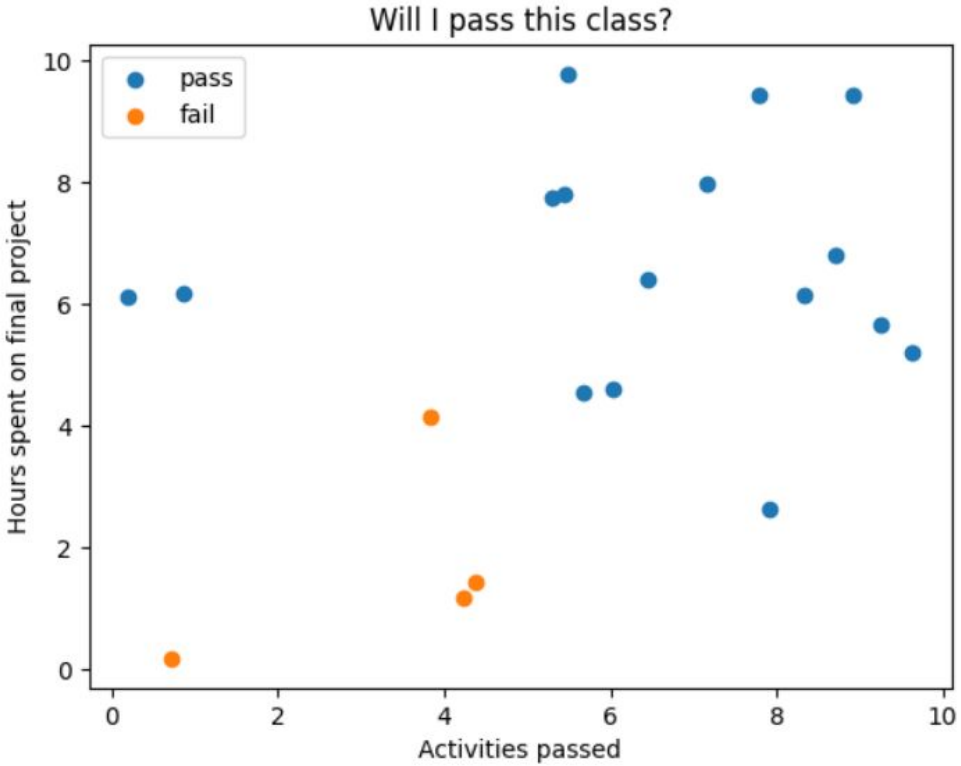
Will I pass this class?

Let's start with a 2 feature model

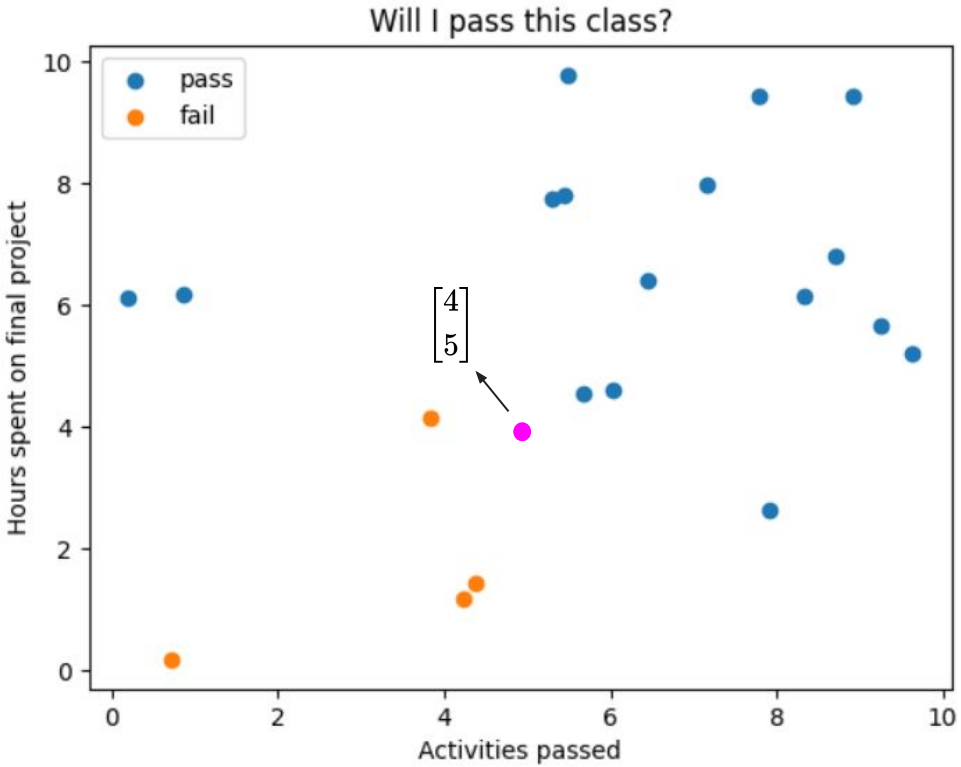
x_1 = Number of activities and assignments you passed

x_2 = Hours spent on final project

Example Problem: Will I pass this class?

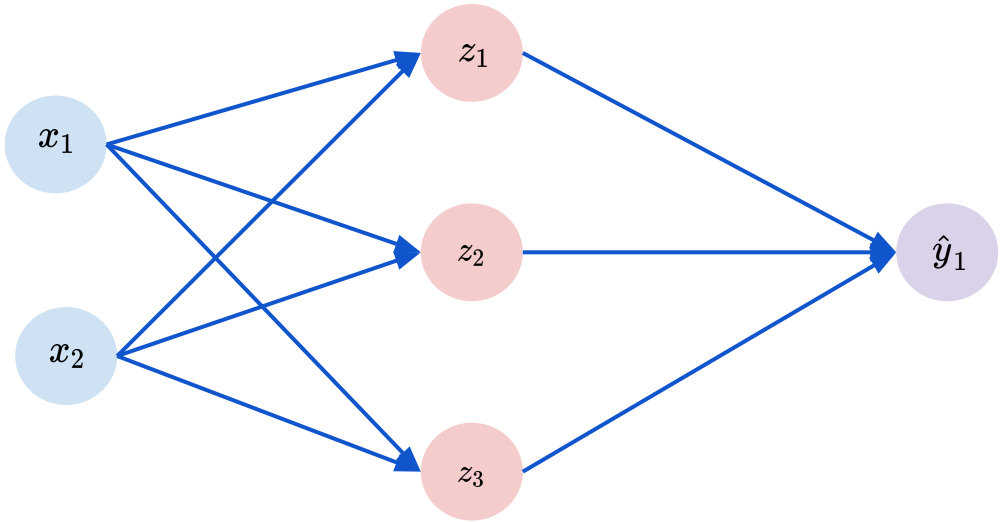


Example Problem: Will I pass this class?



Example Problem: Will I pass this class?

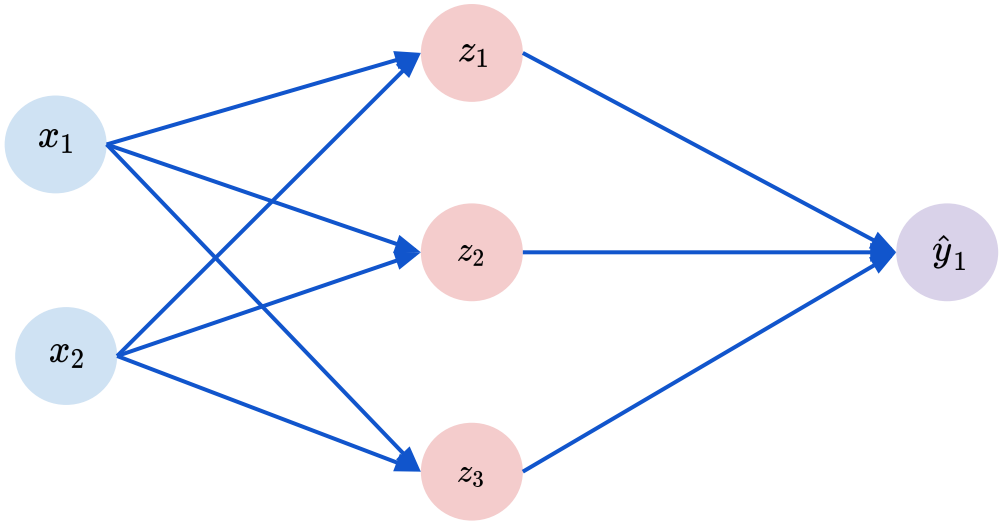
$$\begin{bmatrix} 4 \\ 5 \end{bmatrix}$$



Predicted: 0.1

Example Problem: Will I pass this class?

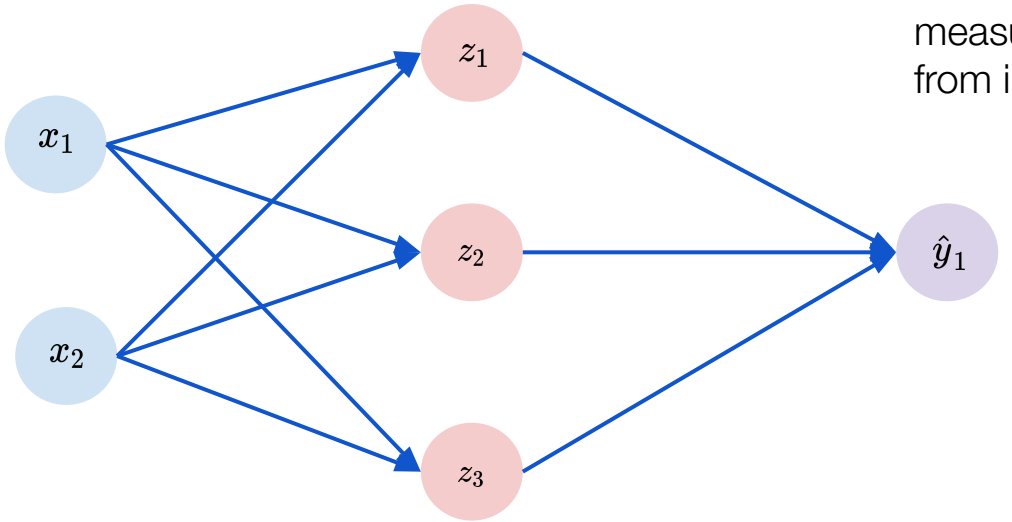
$$\begin{bmatrix} 4 \\ 5 \end{bmatrix}$$



Predicted: 0.1
Actual: 1.0

Quantifying Loss

$$\begin{bmatrix} 4 \\ 5 \end{bmatrix}$$



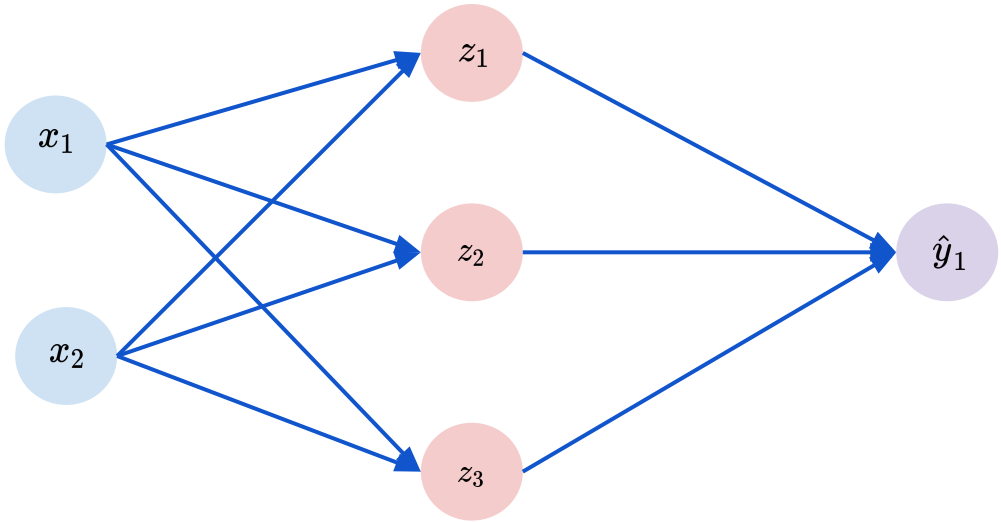
The **loss** of our network measures the cost incurred from incorrect predictions

Predicted: 0.1
Actual: 1.0

Quantifying Loss

The **loss** of our network measures the cost incurred from incorrect predictions

$$\begin{bmatrix} 4 \\ 5 \end{bmatrix}$$



Predicted: 0.1
Actual: 1.0

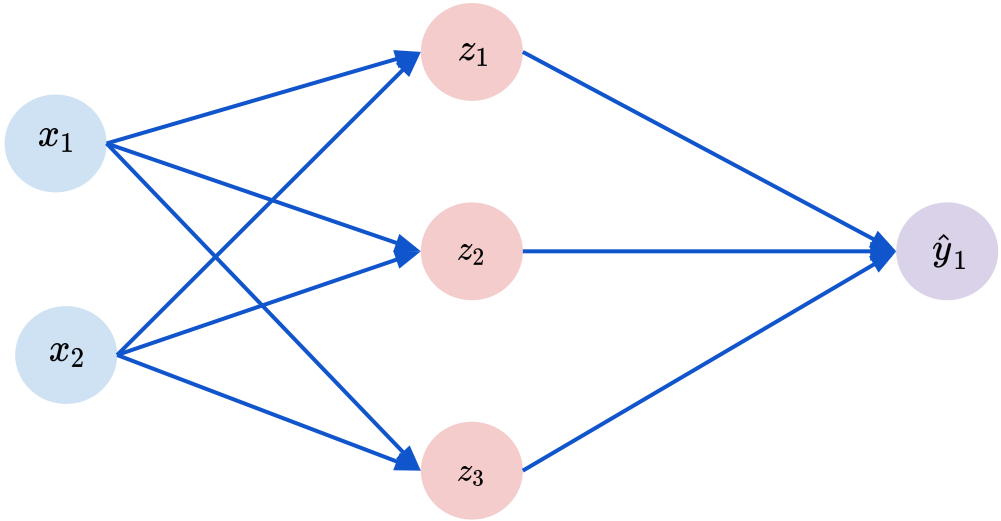
$$L(f(x^{(i)}W), y^{(i)})$$

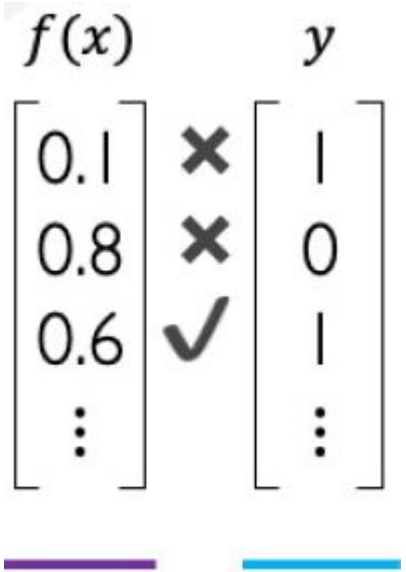
PREDICTED ACTUAL

Empirical Loss

The **empirical loss** measures the total loss over entire dataset

$$\begin{bmatrix} 4 & 2 & 5 & \dots \\ 5 & 1 & 8 & \dots \\ \dots & & & \end{bmatrix}$$





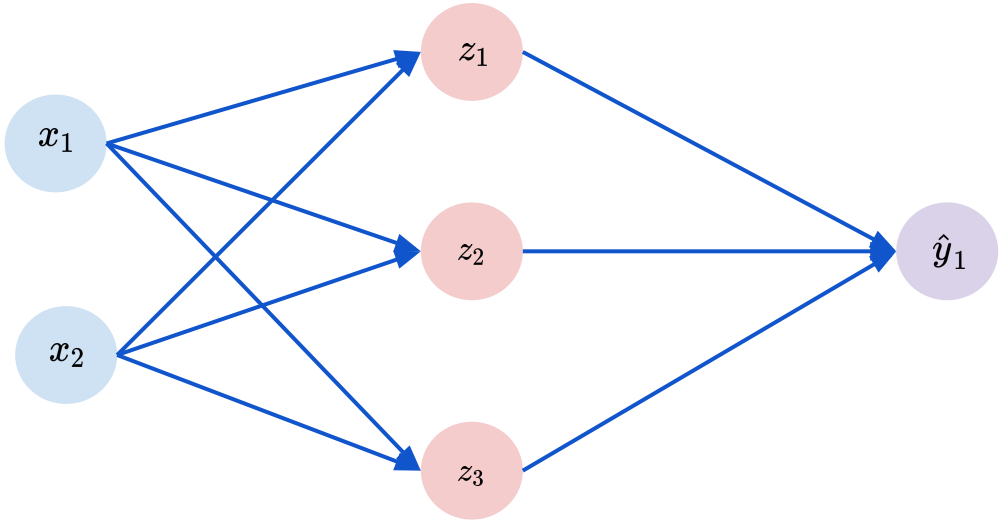
$$J(W) = \frac{1}{n}\sum_{i=1}^{n} L(f(x^{(i)}W), y^{(i)})$$

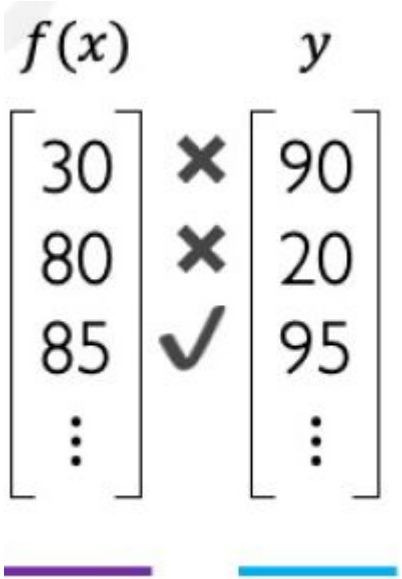
PREDICTED ACTUAL

Mean Squared Error Loss

Mean squared error loss can be used with regression models that output continuous real numbers

$$\begin{bmatrix} 4 & 2 & 5 & \dots \\ 5 & 1 & 8 & \dots \\ \dots & & & \end{bmatrix}$$



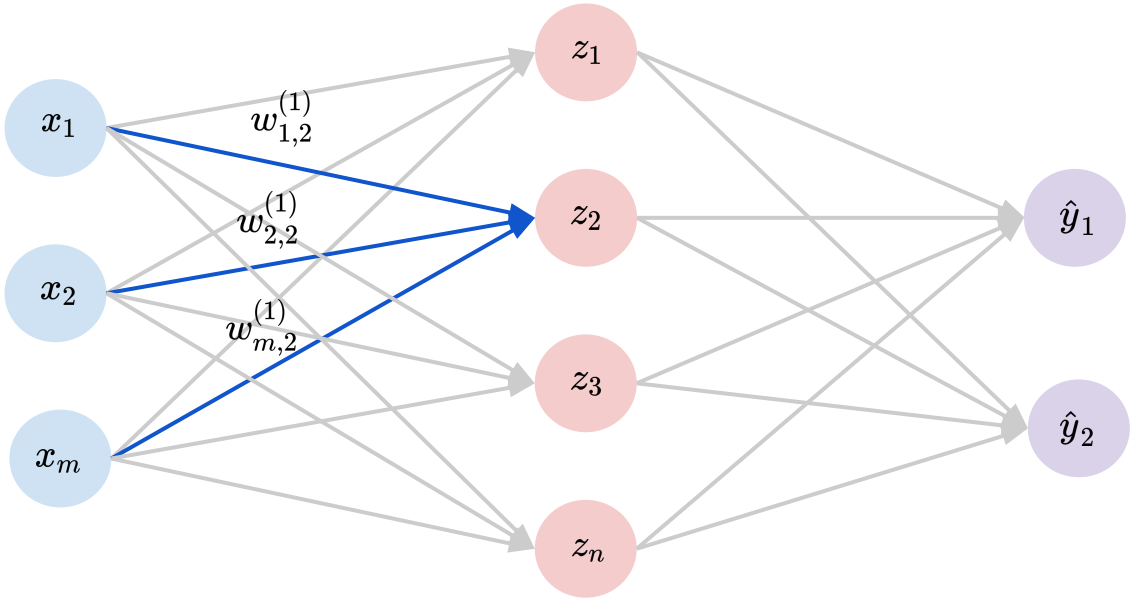


$$J(W) = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - f(x^{(i)}W))^2$$

Training Neural Networks

Loss Optimization

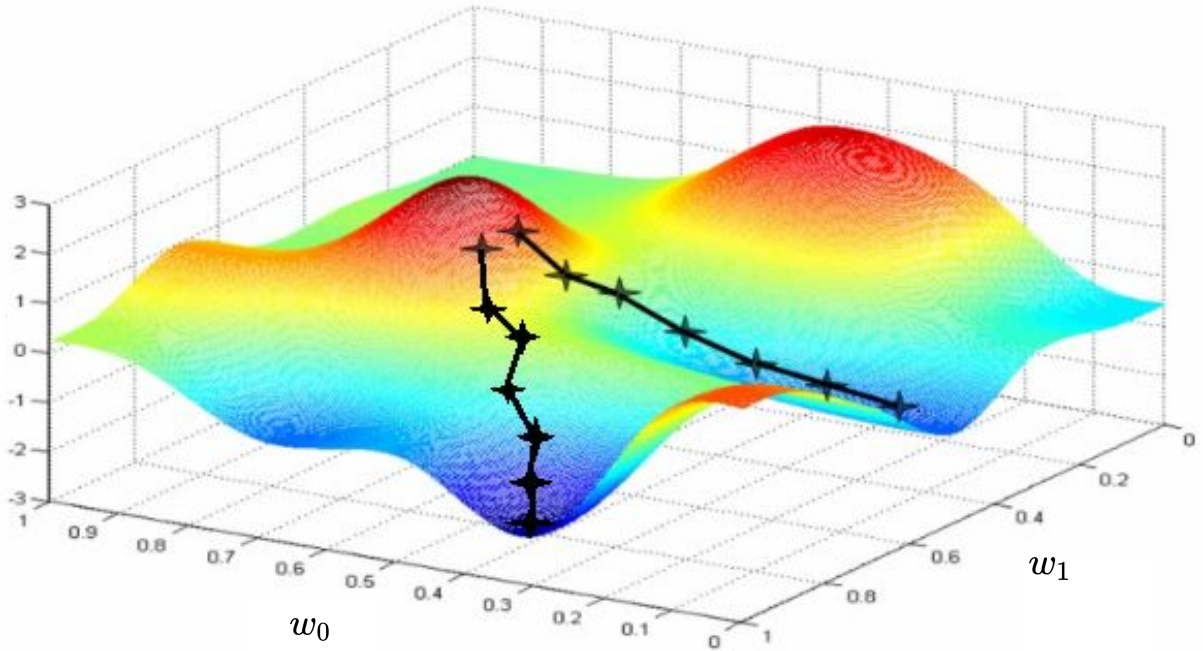
We want to find network weights that achieve the lowest loss



Loss Optimization

We want to find network weights that achieve the lowest loss

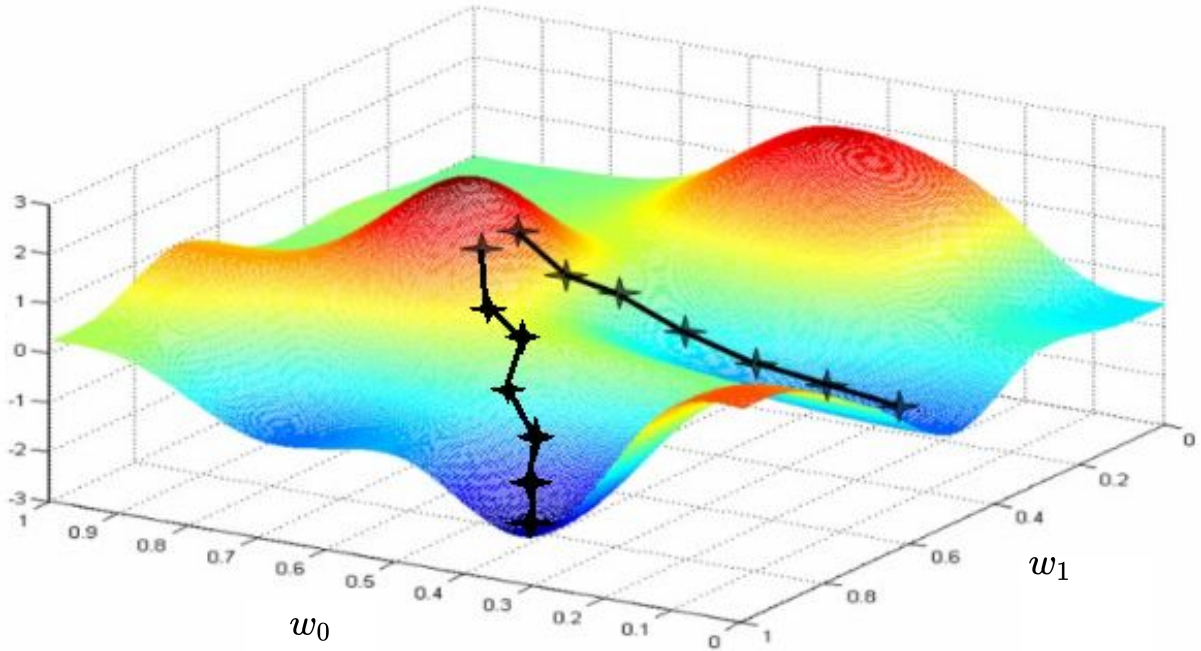
$J(w_0, w_1)$



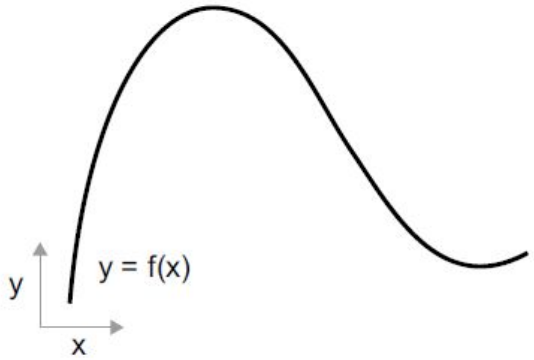
Gradient Descent

The optimization technique that powers modern neural networks.

$J(w_0, w_1)$



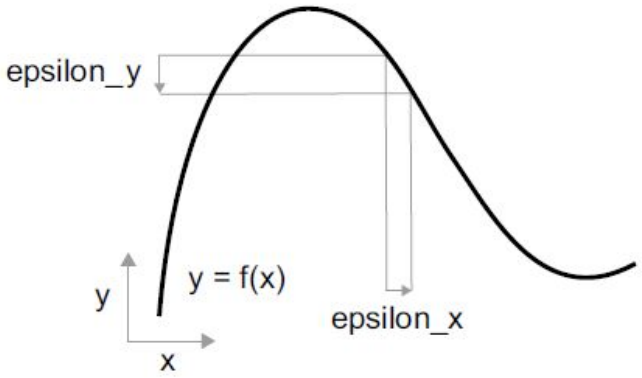
Gradient Descent



Consider a continuous function $f(x) = y$ that maps a number **x** to a new number **y**

A small change in **x** can only result in a small change in **y**

Gradient Descent



Consider a continuous function $f(x) = y$ that maps a number `x` to a new number `y`

A small change in `x` can only result in a small change in `y`

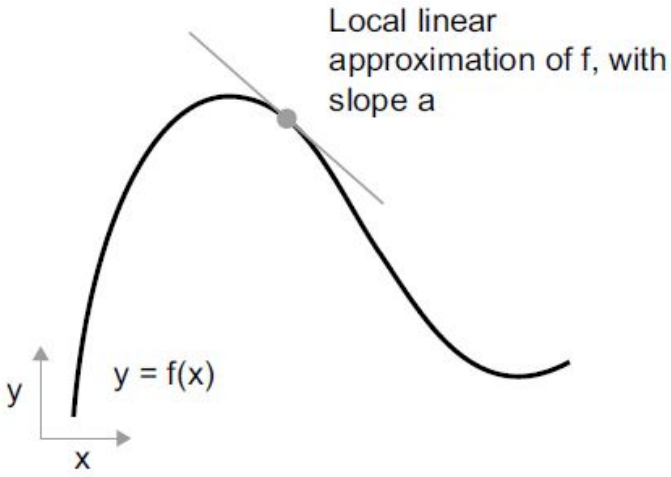
Increase `x` by a small factor `epsilon_x` this results in a small `epsilon_y` change to `y`

When `epsilon_x` is small enough around a certain point `p`, it is possible to approximate `f` as a linear function of a slope `a`, so that

```
f(x + epsilon_x) = y + a * epsilon_x
```

The slope `a` is called the derivative of `f` in `p`.

Gradient Descent



Consider a continuous function $f(x) = y$ that maps a number `x` to a new number `y`

A small change in `x` can only result in a small change in `y`

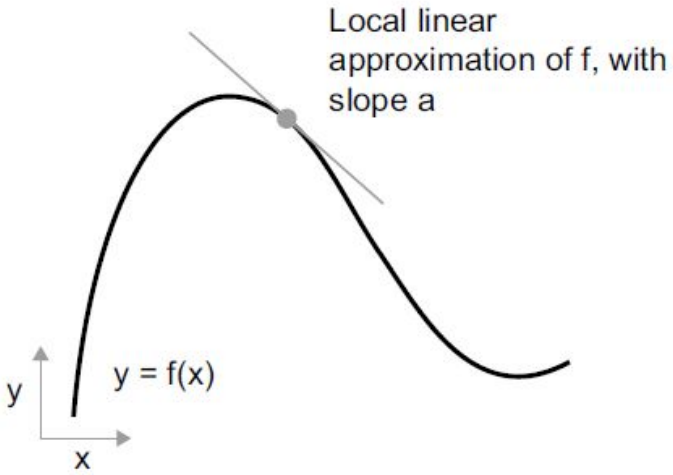
Increase `x` by a small factor `epsilon_x` this results in a small `epsilon_y` change to `y`

When `epsilon_x` is small enough around a certain point `p`, it is possible to approximate `f` as a linear function of a slope `a`, so that

```
f(x + epsilon_x) = y + a * epsilon_x
```

The slope `a` is called the derivative of `f` in `p`.

Gradient Descent



The slope `a` is called the derivative of `f` in `p`.

```
f(x + epsilon_x) = y + a * epsilon_x
```

`a < 0` means a small increase in `x` around `p` will result in in a decrease of `f(x)`

`a > 0` means a small increase in `x` around `p` will result in an increase of `f(x)`

The absolute value of `a`, the magnitude of the derivative, will tell you how quickly the decrease/increase will happen

Gradient Descent

A **gradient** is a vector of partial derivatives.

The partial derivative of a function with respect to a variable is the derivative assuming all other input variables are held constant.

Given the following cost function:

$$J(W) = \frac{1}{n} \sum_{i=1}^{n} (y^{(i)} - f(x^{(i)} W))^2$$

Translated as a linear function,

$$f(m, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (mx_i + b))^2$$

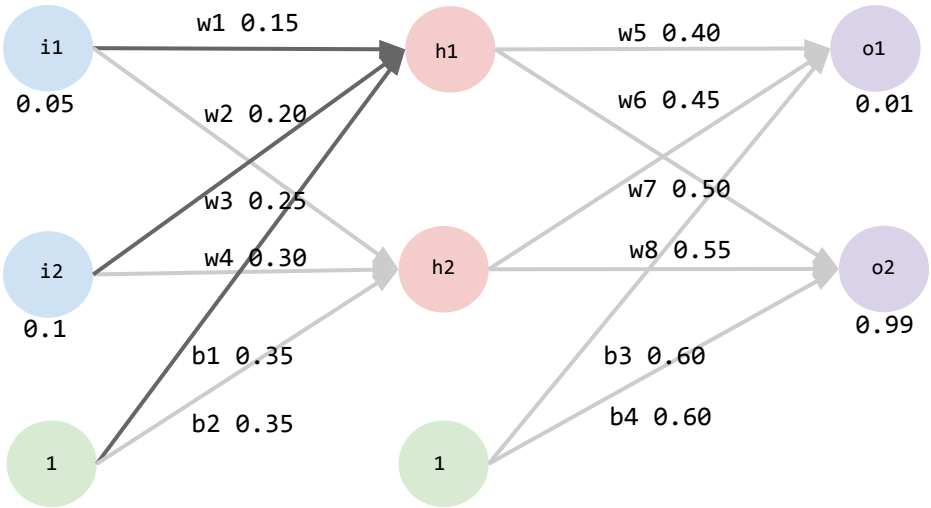
Gradient Descent

The gradient can be calculated as,

$$f'(m,b) = \begin{bmatrix} \frac{df}{dm} \\ \frac{df}{db} \end{bmatrix} = \begin{bmatrix} \frac{1}{n} \sum -2x_i(y_i - (mx_i + b)) \\ \frac{1}{n} \sum -2(y_i - (mx_i + b)) \end{bmatrix}$$

We iterate through our data points using our new m and b values and compute the partial derivatives. This new gradient tells us the slope of our cost function at our current position (current parameter values) and the direction we should move to update our parameters.

Forward Propagation



Compute Hidden Layer Activations:

Calculate the net input to each hidden layer unit

```
z1 = (i1 * w1) + (i2 * w3) + b1
   = (0.05 * 0.15) + (0.10 * 0.25) + 0.35
   =  0.3824

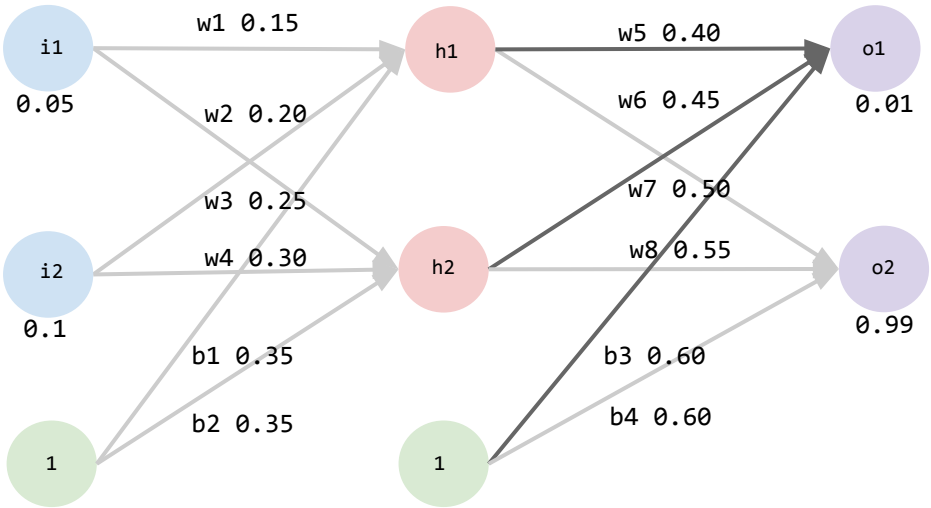
z2 = (i1 * w2) + (i2 * w4) + b2
   = (0.05 * 0.20) + (0.10 * 0.30) + 0.35
   =  0.3899
```

Squash the net input using an activation function

$h1 = \frac{1}{1+e^{-z_1}} = 0.5944$

$h2 = \frac{1}{1+e^{-z_2}} = 0.5962$

Forward Propagation



Compute Output Layer Activations:

Calculate the net input to each output layer unit

```
z3 = (h1 * w5) + (h2 * w7) + b3
   = (0.5944 * 0.40) + (0.5962 * 0.50) + 0.60
   =  1.13586

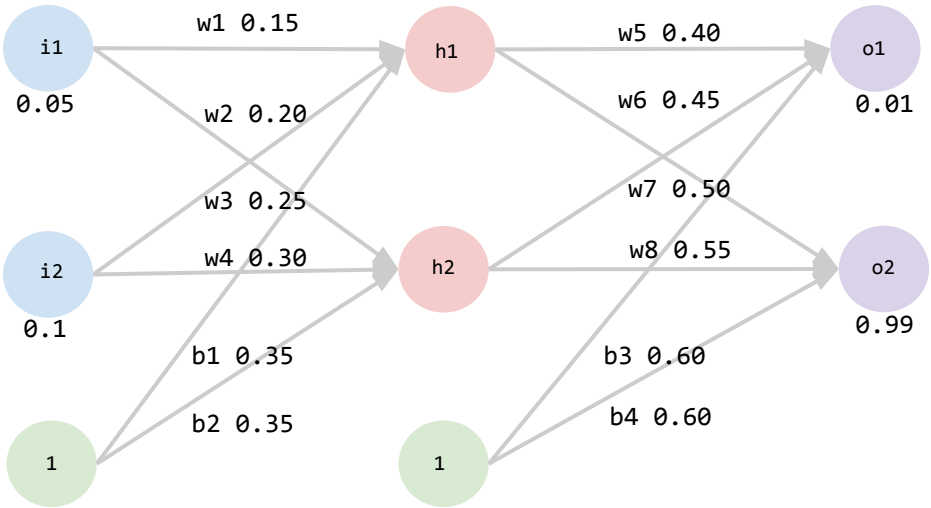
z4 = (h1 * w6) + (h2 * w8) + b4
   = (0.5944 * 0.45) + (0.5962 * 0.55) + 0.60
   =  1.1953
```

Squash the net input using an activation function

o1 = $\frac{1}{1+e^{-z_3}}$ = 0.7569

o2 = $\frac{1}{1+e^{-z_4}}$ = 0.7676

Forward Propagation



Compute Output Layer Errors:

Calculate the errors at the output layer

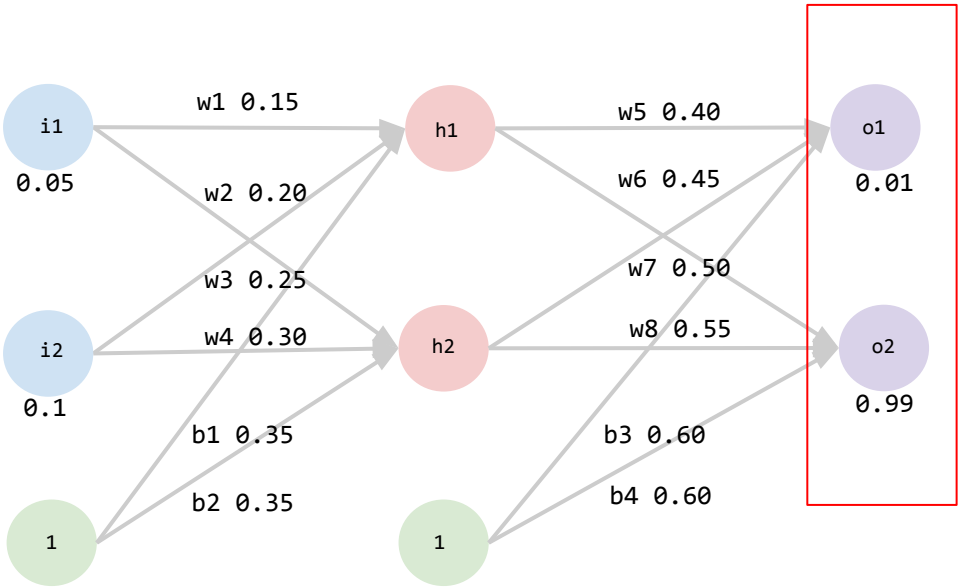
$$\frac{1}{2}\sum(actual - predicted)^2$$

$$E = 0.5 *((0.01 - 0.7569)^2 + (0.99 - 0.7729)^2)$$
$$= 0.5 *(0.5578 + 0.0471)$$
$$= 0.5 * 0.6049$$
$$= 0.30245$$

Update each of the weights in the network so the predicted output will be closer to the actual output.

Minimizing the error of each output neuron.

Backward Propagation

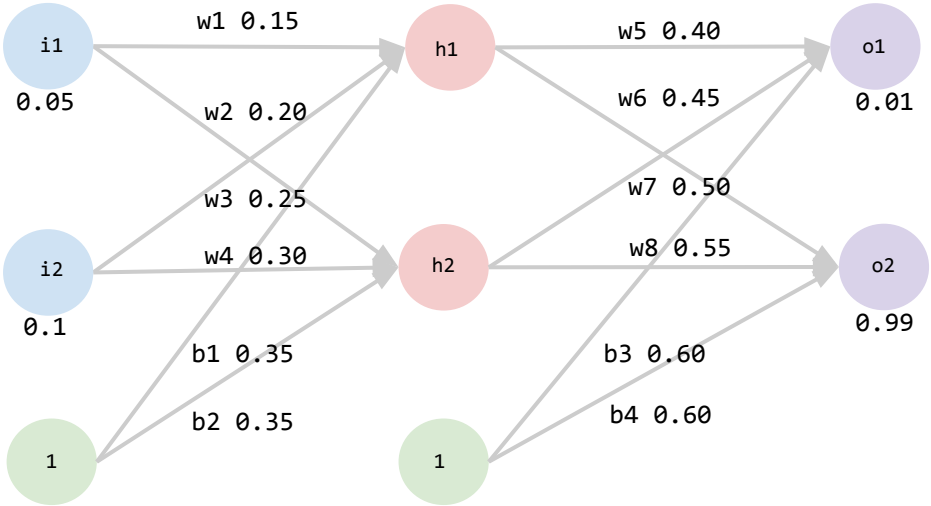


Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

Consider `w5`,

$$\frac{dE}{dw5} = \frac{dE}{do1} * \frac{do1}{dz3} * \frac{dz3}{dw5}$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

Consider `w5`,

$$\frac{dE}{dw5} = \frac{dE}{do1} * \frac{do1}{dz3} * \frac{dz3}{dw5}$$

First, how much the total error change with respect to the output?

$$E = \frac{1}{2}(actual_{o1} - predicted_{o1})^2 + \frac{1}{2}(actual_{o2} - predicted_{o2})^2$$

$$\frac{dE}{do1} = 2 * \frac{1}{2}(actual_{o1} - predicted_{o1})^{2-1} * -1 + 0$$

$$\frac{dE}{do1} = -(actual_{o1} - predicted_{o1})$$

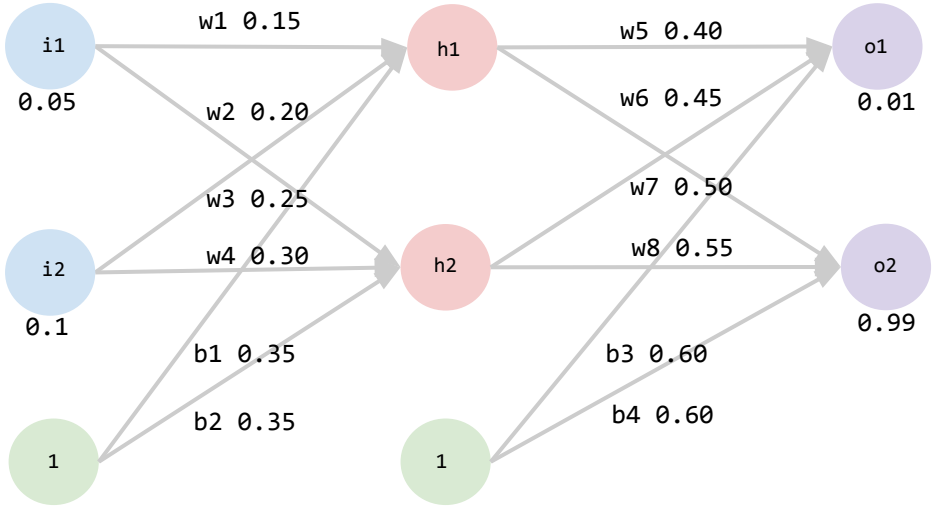
```
= -(0.01 - 0.7569)
= -(-0.7469)
= 0.7469
```

`2` comes from the power rule of differentiation. When you differentiate `(actual-predicted)`^{2} with respect to `predicted` the exponent decreases by `1`.

`-1` comes from chain rule when taking the derivative of `E` with respect to `predicted`.

The chain rule requires you to multiply the derivative of the outer function `E` by the derivative of the inner function `(actual - predicted)` with respect to `predicted`. The derivative of `-predicted` with respect to `predicted` is `-1`

Backward Propagation



When we take the partial derivative of the total error `E` with respect `o1` the quantity `½(actual`$_{o2}$ `- predicted`$_{o2}$`)`^{2} becomes `0` because `o1` does not affect it.

It means we are taking the derivative of a constant which is `0`.

Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

Consider `w5`,

$$\frac{dE}{dw5} = \frac{dE}{do1} * \frac{do1}{dz3} * \frac{dz3}{dw5}$$

First, how much the total error change with respect to the output?

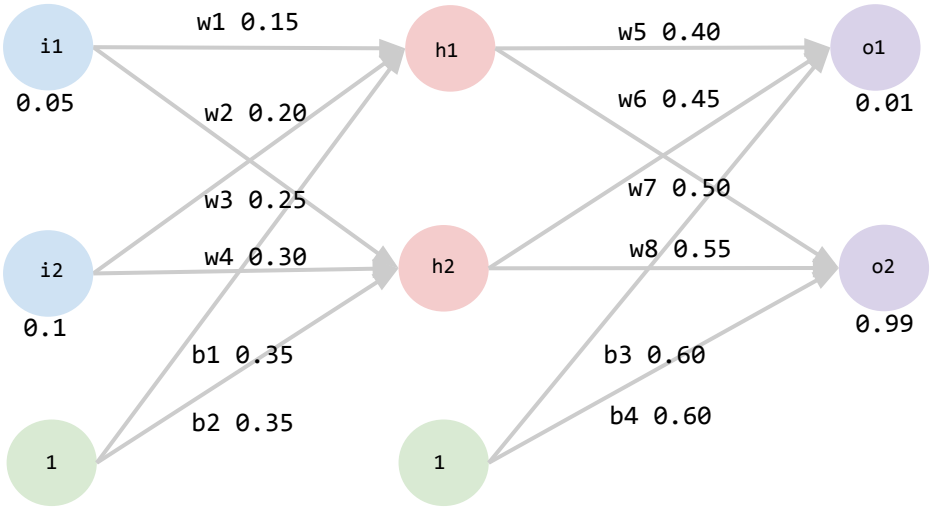
$$E = \frac{1}{2}(actual_{o1} - predicted_{o1})^2 + \frac{1}{2}(actual_{o2} - predicted_{o2})^2$$

$$\frac{dE}{do1} = 2 * \frac{1}{2}(actual_{o1} - predicted_{o1})^{2-1} * -1 + 0$$

$$\frac{dE}{do1} = -(actual_{o1} - predicted_{o1})$$

```
= -(0.01 - 0.7569)
= -(-0.7469)
= 0.7469
```

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

Consider `w5`,

$$\frac{dE}{dw5} = \frac{dE}{do1} * \frac{do1}{dz3} * \frac{dz3}{dw5}$$

Next, how much does `o1` change with respect to its total input?

`o1` = $\frac{1}{1+e^{-z_3}}$

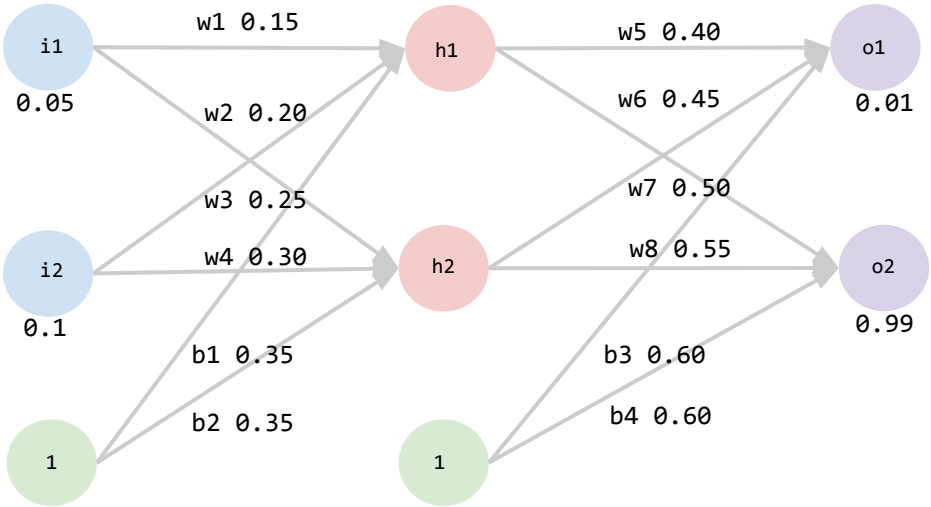
$$\frac{do1}{dz3} = \texttt{o1 * (1 - o1)}$$

```
= 0.7569 * (1 - 0.7569)
= 0.1840
```

Calculate the derivative of the sigmoid using the quotient rule.

http://en.wikipedia.org/wiki/Logistic_function#Derivative

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

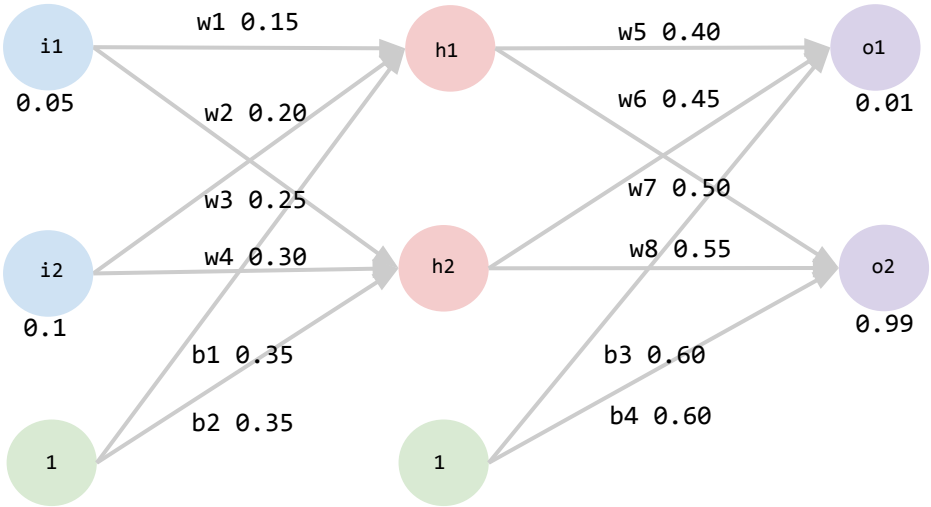
Consider `w5`,

$$\frac{dE}{dw5} = \frac{dE}{do1} * \frac{do1}{dz3} * \frac{dz3}{dw5}$$

Next, how much the total input `z3` change with respect to `w5`?

```
z3 = (h1 * w5) + (h2 * w7) + b3 * 1
   = h1 + 0 + 0
   = h1
   = 0.5944
```

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

Consider `w5`,

$$\frac{dE}{dw5} = \frac{dE}{do1} * \frac{do1}{dz3} * \frac{dz3}{dw5}$$

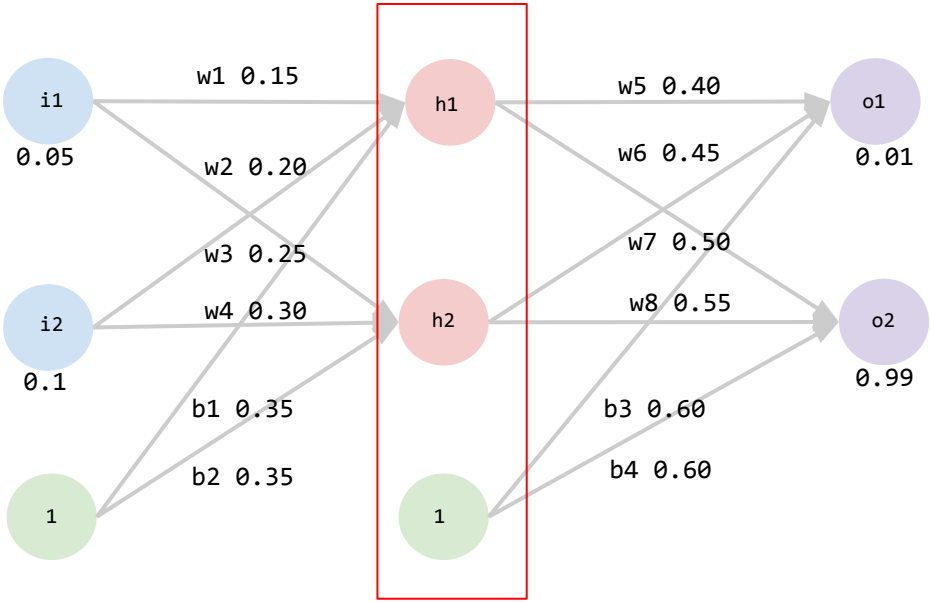
Putting it all together,

$$\frac{dE}{dw5} = 0.7469 * 0.1840 * 0.5944 = 0.0816$$

To decrease the error, we subtract the value from the current weight together with a learning rate (sometimes expressed in various literature as alpha α, eta η, epsilon ϵ)

```
w5 = 0.40 - (0.3 * 0.0816) = 0.3755
```

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider **w1**,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

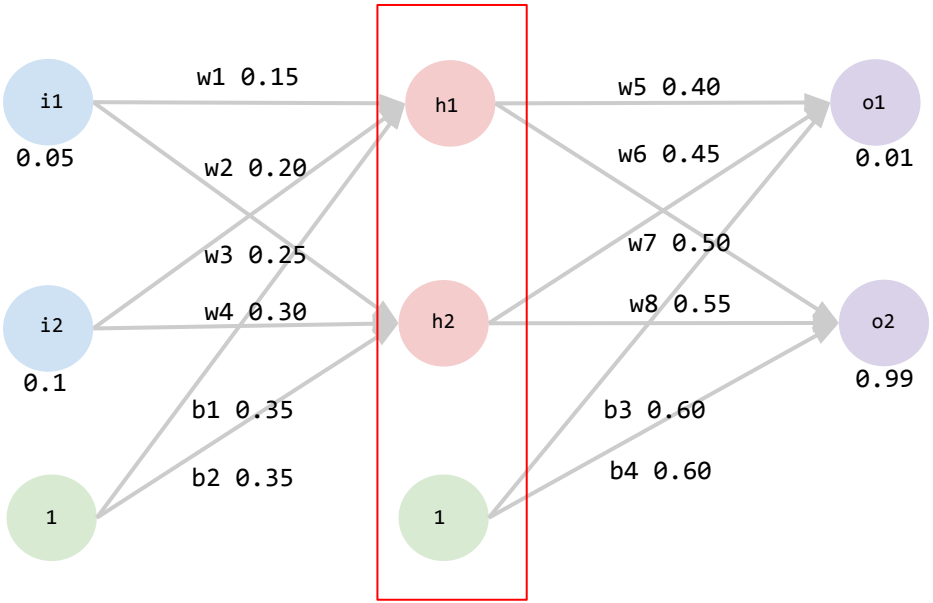
Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1}$$

Since the output of each hidden layer neuron contributes to the output (and error) of multiple output neurons.

The output **h1** affects both **o1** and **o2** therefore we need to consider its effect on both output neurons.

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider **w1**,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

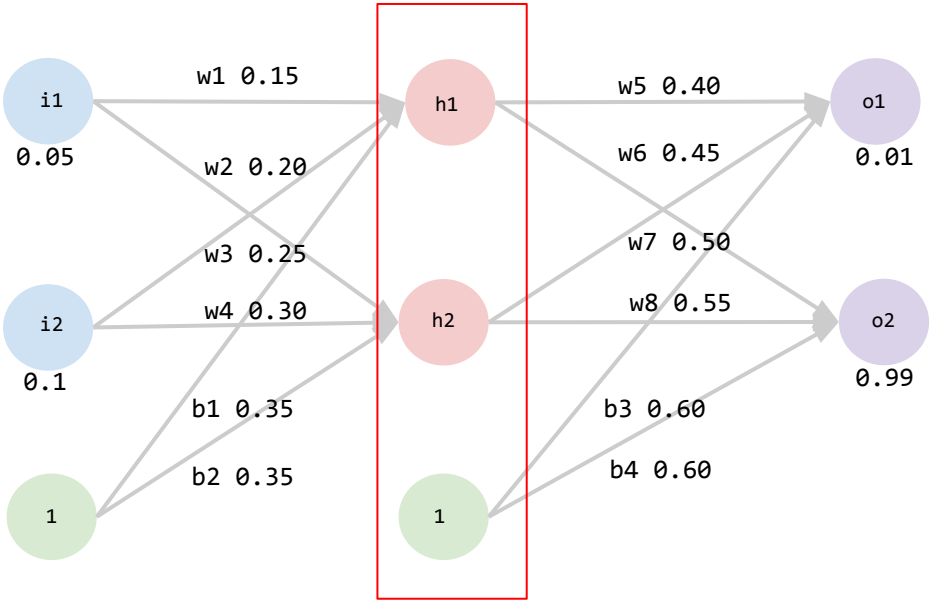
Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1}$$

Starting with $\frac{dEo1}{dh1}$,

$$\frac{dEo1}{dh1} = \frac{dEo1}{dz3} * \frac{dz3}{dh1}$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider `w1`,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1}$$

Starting with $\frac{dEo1}{dh1}$,

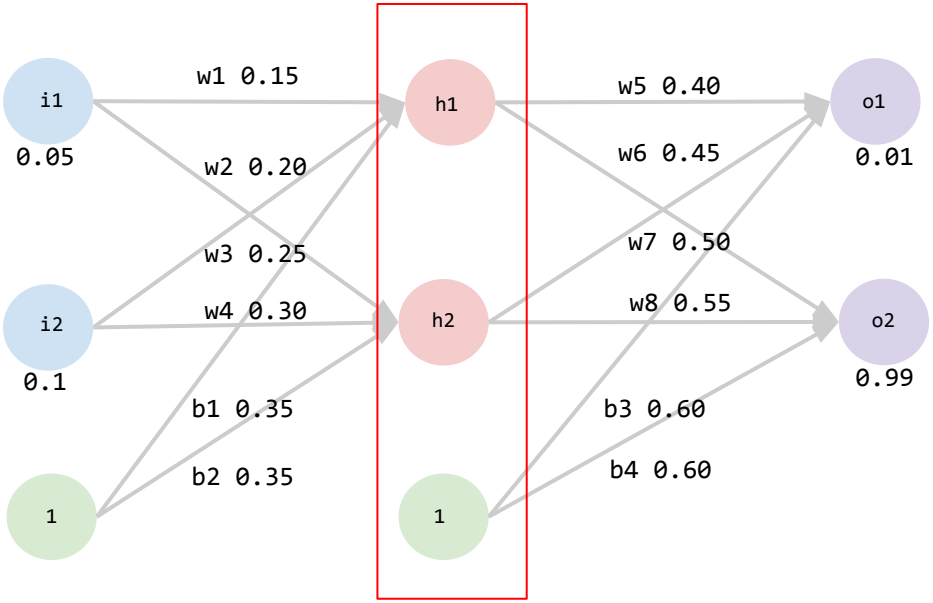
$$\frac{dEo1}{dh1} = \frac{dEo1}{dz3} * \frac{dz3}{dh1}$$

We can calculate $\frac{dEo1}{dz3}$ and $\frac{dz3}{dh1}$,

$$\frac{dEo1}{dz3} = \frac{dEo1}{do1} * \frac{do1}{dz3}$$

$$\frac{dz3}{dh1} = \texttt{(h1 * w5) + (h2 * w7) + b3}$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider `w1`,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1}$$

Starting with $\frac{dEo1}{dh1}$,

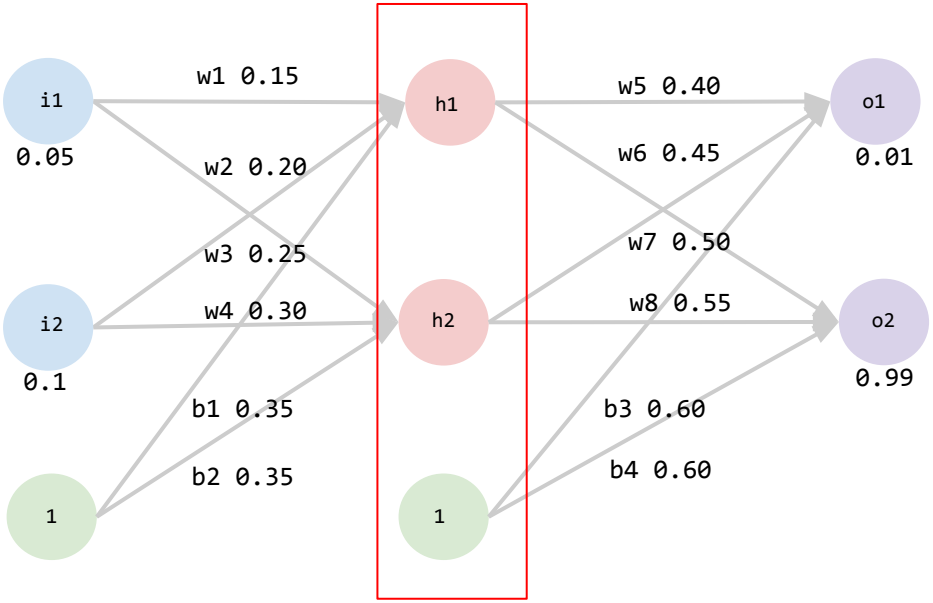
$$\frac{dEo1}{dh1} = \frac{dEo1}{dz3} * \frac{dz3}{dh1}$$

We can calculate $\frac{dEo1}{dz3}$ and $\frac{dz3}{dh1}$,

$$\frac{dEo1}{dz3} = \frac{dEo1}{do1} * \frac{do1}{dz3} = 0.7469 * 0.1840 = 0.1374$$

$$\frac{dz3}{dh1} = (h1 * w5) + (h2 * w7) + b3 = w5 = 0.40$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider `w1`,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1}$$

Starting with $\frac{dEo1}{dh1}$,

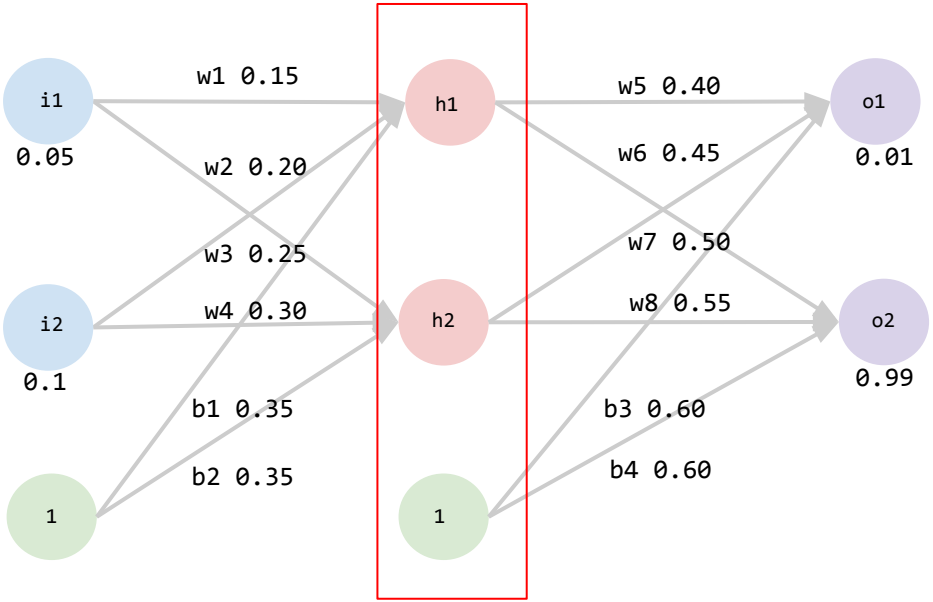
$$\frac{dEo1}{dh1} = \frac{dEo1}{dz3} * \frac{dz3}{dh1} = 0.1374 * 0.40 = 0.0549$$

We can calculate $\frac{dEo1}{dz3}$ and $\frac{dz3}{dh1}$,

$$\frac{dEo1}{dz3} = \frac{dEo1}{do1} * \frac{do1}{dz3} = 0.7469 * 0.1840 = 0.1374$$

$$\frac{dz3}{dh1} = (h1 * w5) + (h2 * w7) + b3 = w5 = 0.40$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider **w1**,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1}$$

Doing the same for $\frac{dEo2}{dh1}$,

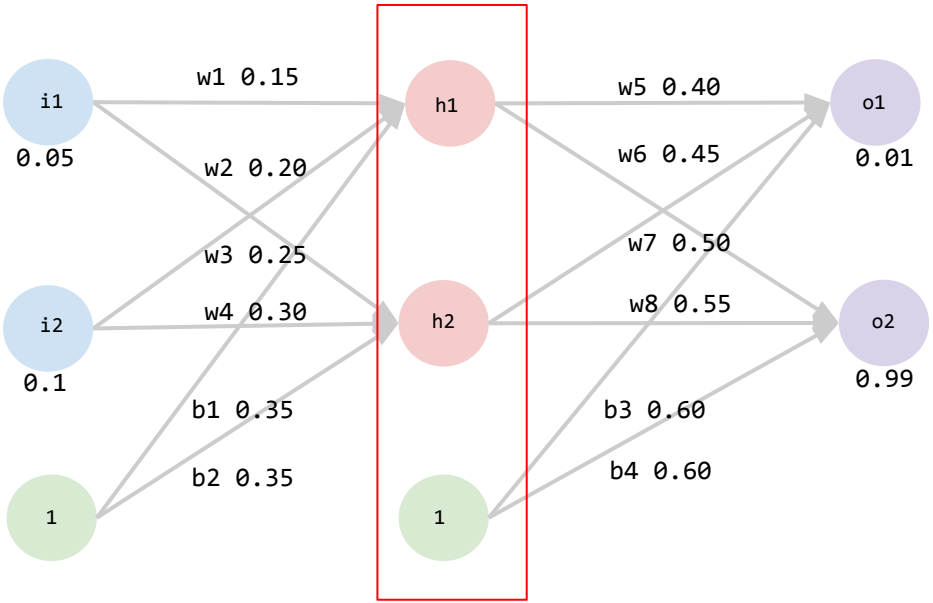
$$\frac{dEo2}{dh1} = \frac{dEo2}{dz4} * \frac{dz4}{dh1} = \texttt{-0.0399 * 0.50 = -0.0199}$$

We can calculate $\frac{dEo2}{dz4}$ and $\frac{dz4}{dh1}$,

$$\frac{dEo2}{dz4} = \frac{dEo2}{do2} * \frac{do2}{dz4} = \texttt{-0.2224 * 0.1796 = -0.0399}$$

$$\frac{dz4}{dh1} = \texttt{(h1 * w7) + (h2 * w8) + b4 = w7 = 0.50}$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider `w1`,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

Where $\frac{dE}{dh1}$,

$$\frac{dE}{dh1} = \frac{dEo1}{dh1} + \frac{dEo2}{dh1} = 0.0549 + (-0.0199) = 0.0349$$

Doing the same for $\frac{dEo2}{dh1}$,

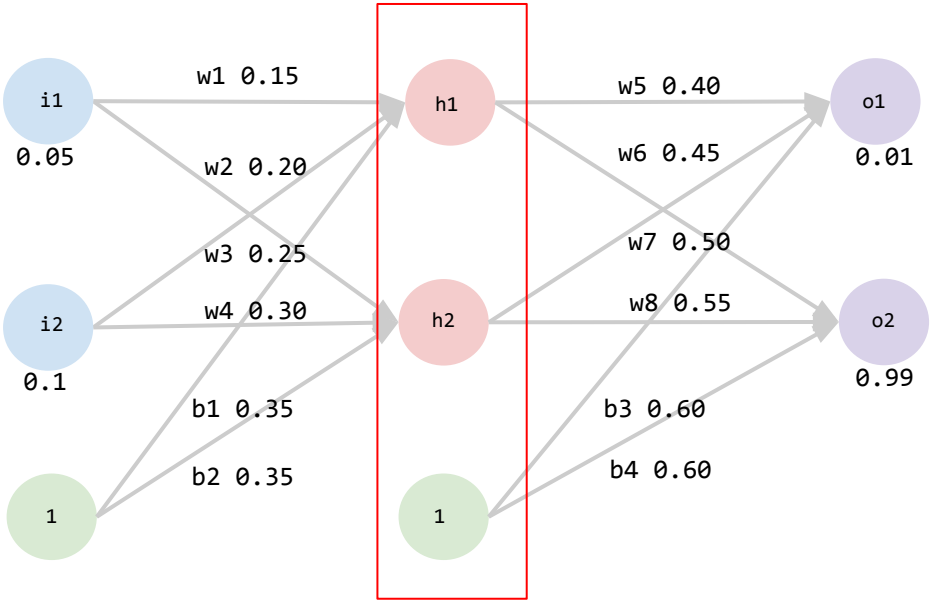
$$\frac{dEo2}{dh1} = \frac{dEo2}{dz4} * \frac{dz4}{dh1} = -0.0399 * 0.50 = -0.0199$$

We can calculate $\frac{dEo2}{dz4}$ and $\frac{dz4}{dh1}$,

$$\frac{dEo2}{dz4} = \frac{dEo2}{do2} * \frac{do2}{dz4} = -0.2224 * 0.1796 = -0.0399$$

$$\frac{dz4}{dh1} = (h1 * w7) + (h2 * w8) + b4 = w7 = 0.50$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

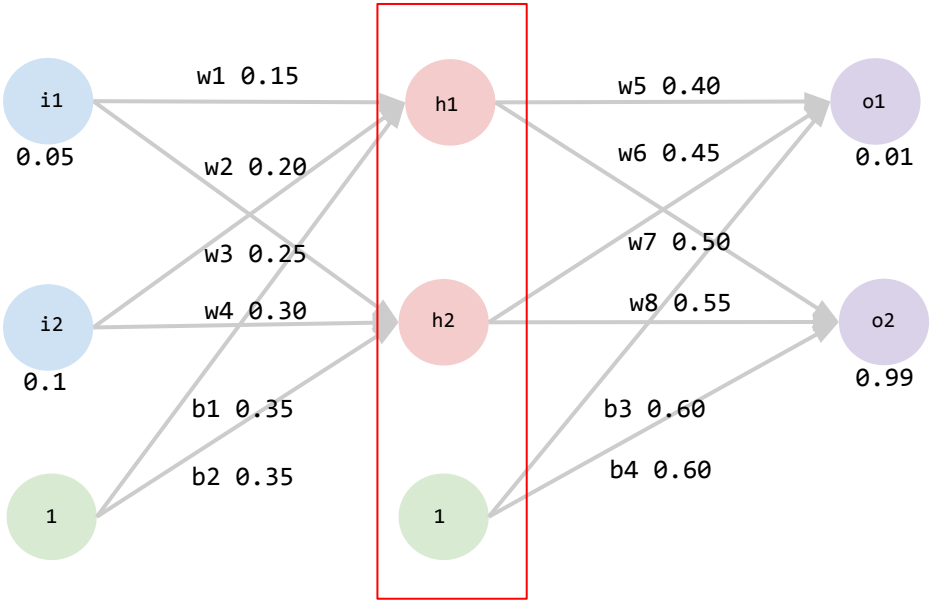
We will continue with the backward pass by calculating new weights for the hidden layer. Consider **w1**,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

Where $\frac{dh1}{dz1}$ and **h1** = $\frac{1}{1+e^{-z_1}}$,

$\frac{dh1}{dz1}$ = `h1 * (1 - h1) = 0.5944 * (1 - 0.5944) = 0.2410`

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

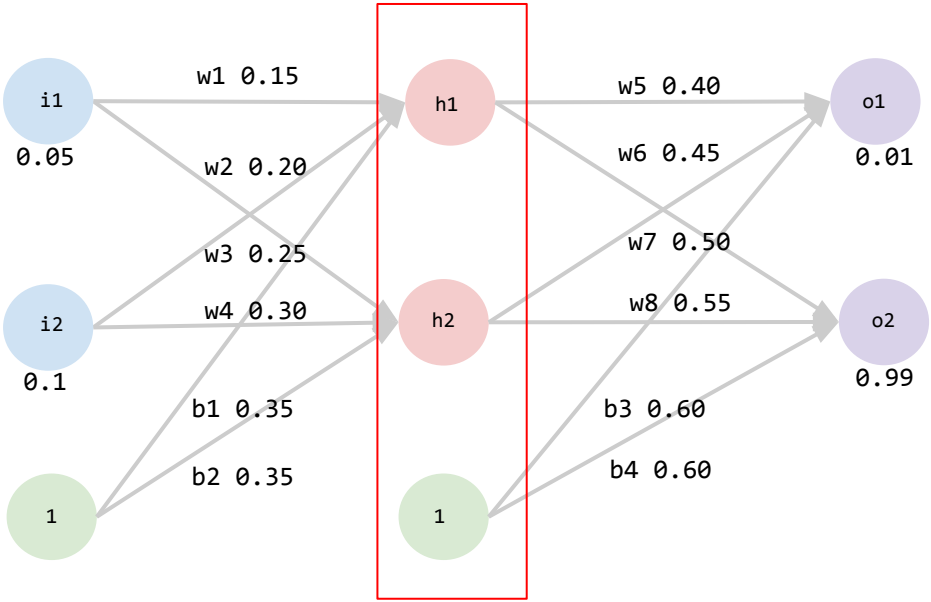
We will continue with the backward pass by calculating new weights for the hidden layer. Consider **w1**,

$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

and $\frac{dz1}{dw1}$,

$$\frac{dz1}{dw1} = \texttt{(i1 * w1) + (i2 * w3) + b1 = i1 = 0.05}$$

Backward Propagation



Compute the partial derivatives or gradients with respect to their activations by applying the chain rule:

We will continue with the backward pass by calculating new weights for the hidden layer. Consider **w1**,

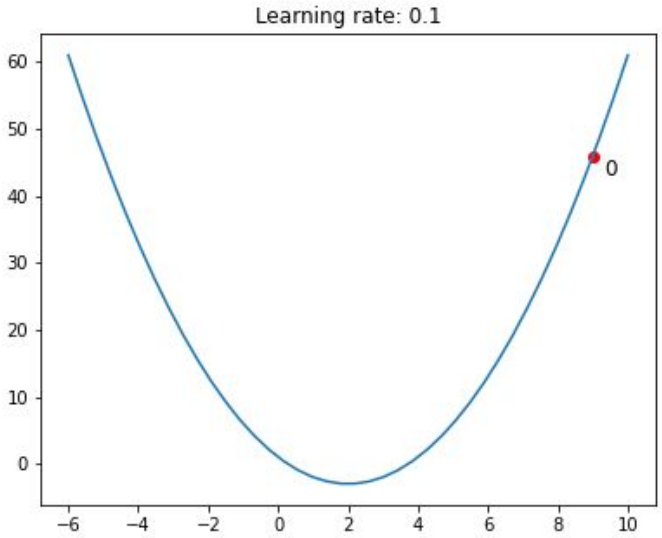
$$\frac{dE}{dw1} = \frac{dE}{dh1} * \frac{dh1}{dz1} * \frac{dz1}{dw1}$$

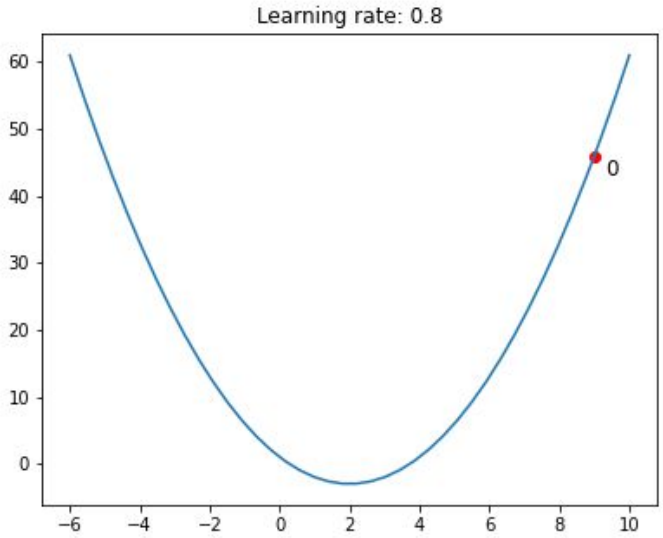
$$= \texttt{0.0349 * 0.2410 * 0.05 = 0.0004}$$

We can now update **w1**, we subtract the value from the current weight together with a learning rate

```
w1 = 0.15 - (0.3 * 0.0004) = 0.1498
```

Learning Rate

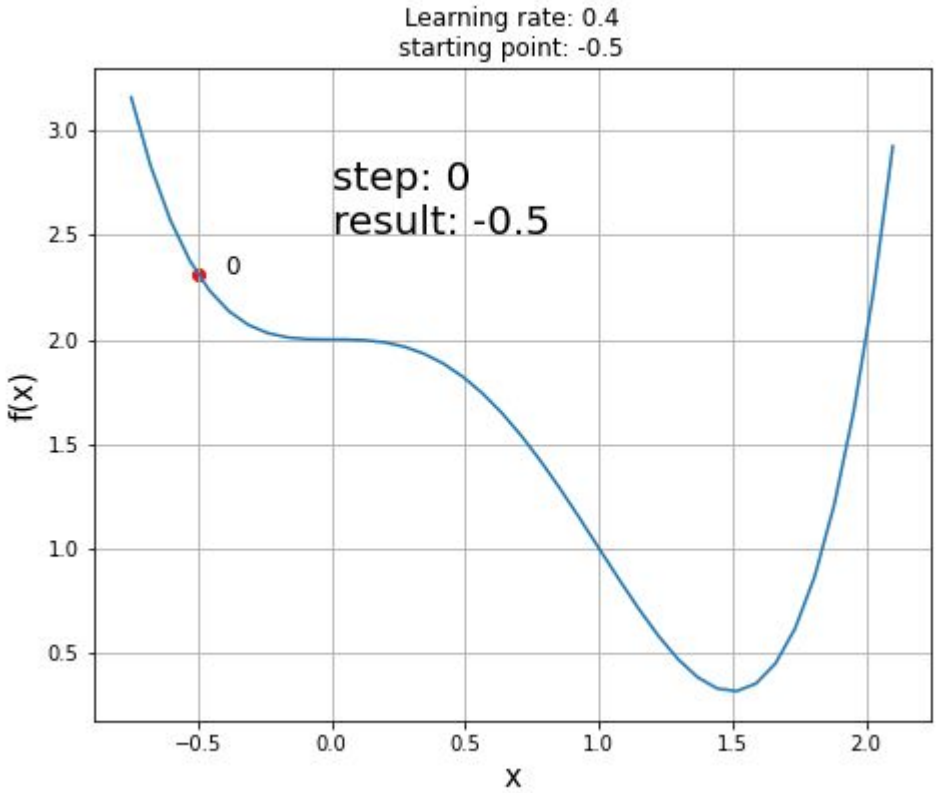




Recall that we update the weights so we can move the weights by “a bit” in a direction that will reduce the loss. The magnitude of the move is determined by a small quantity called **learning rate**.

Learning rate is the size of the steps we take towards a local minimum.

Learning Rate



- If learning rate is too small, Gradient Descent takes longer to converge or may even reach maximum iteration before reaching an optimal point
- If learning rate is too big, Gradient descent may not converge to an optimal point, overshoot, or diverge completely.